

Projekt Dokumentation
Labyrinth Maus

Benjamin Hage

Matr,. Nr.: 299912

Studiengang: EIM

Inhaltsverzeichnis

Einleitung.....	5
Installationsanleitung.....	6
1. Raspberry Pi OS installieren	6
2. Repository klonen.....	6
3. Notwendige Python-Bibliotheken installieren	6
4. I2C-Schnittstelle aktivieren und testen	6
5. ABElectronics-Python-Bibliotheken installieren	6
6. Motoron-Bibliothek installieren.....	7
7. GPIO-Bibliothek für Raspberry Pi installieren	7
8. Adafruit IMU-Treiber installieren.....	7
Hardware.....	8
Der Roboter	8
Antriebsmodul.....	8
Sensormodul	9
Steuerungsmodul	9
Akkumodul	10
Das Labyrinth.....	10
Begrifflichkeiten	11
Programmnutzung.....	12
1. Starten des Programms	12
2. Verfügbare Befehlszeilenparameter	12
3. Manuelle Steuerung.....	12
4. Automatikmodus.....	13
5. Fehlerbehebung	13
Konsolenausgabe	14
1. Allgemeine Struktur der Konsolenausgabe	14
2. Bedeutung der einzelnen Werte	14
3. Automatikmodus.....	15
Dokumentation des Zustandsautomaten und des Pledge-Algorithmus.....	16
Zustandsautomat	16
Zustände und Zustandsübergänge	20

Zustand 0: Initialisierung	20
Zustand 1: Linker Wand folgen	21
Zustand 2: Rechter Wand folgen.....	21
Zustand 3: Vorbereitung auf 90°-Drehung nach rechts	22
Zustand 4: Vorbereitung auf 90°-Drehung nach links	22
Zustand 5: Geregelte Drehung	22
Zustand 6: Geradeausfahren	24
Zustand 7: Zielpunkt setzen	26
Zustand 8: Regelung zum Zielpunkt	26
Zustand 9: Regelung zur vorderen Wand.....	27
Zustand 10: Vorbereitung auf 45°-Drehung nach rechts	28
Zustand 11: Vorbereitung auf 45°-Drehung nach links.....	28
Zustand 12: Vorbereitung auf 0°-Drehung.....	29
Zustand 13: ESC (Extremum Seeking Control)	29
Zustand 14: Ungeregelt drehen	29
Zustand 15: Vorbereitung auf 360°-Drehung.....	30
Zustand 16: (Reserviert)	30
Zustand 17: (Reserviert)	30
Zustand 18: Vorbereitung PID-Regelung für linke Wand	31
Zustand 19: PID-Regelung für rechte Wand.....	31
Zustand 20: Winkel mit kleinstem Abstand einstellen.....	32
Zustand 21: Links diagonal folgen	32
Zustand 22: Rechts diagonal folgen	33
Dokumentation der Dateien	35
Controller.py.....	35
Klasse PIDController	35
Klasse ESCController.....	37
Klasse AutonomousController.....	39
filter.py	45
Klasse LowPassFilter	45
Klasse HighPassFilter	46
Klasse UKFEstimator	46

lo_management.py	48
Klasse OutputManager	48
Klasse ConsoleOutput	50
Klasse RealTimePlotter	53
Funktion: handle_user_input	54
robot.py	56
Klasse DifferentialDriveRobot	56
Maus.py	69
Kommandozeilenparameter	69
Steuerung	69
Autonome Steuerung	70
Ausgabe	73
Ablauf der Hauptschleife	73
Probleme und Verbesserungsmöglichkeiten	78

Einleitung

Diese Projektarbeit beschreibt die Entwicklung und Steuerung eines autonomen Labyrinth-Roboters. Ziel des Projekts ist es, einen Roboter zu konstruieren, der selbstständig den Ausgang aus einem Labyrinth findet. Der Roboter wird von einem Raspberry Pi gesteuert und die Software ist in Python geschrieben.

Die Arbeit befasst sich ausführlich mit verschiedenen Aspekten der Steuerungssoftware, den Methoden der autonomen Steuerung sowie den Problemen und Lösungen, die bei der Entwicklung entstanden sind.

Zudem wird eine detaillierte Installationsanleitung für das notwendige Betriebssystem und die erforderlichen Bibliotheken bereitgestellt, um den Roboter erfolgreich in Betrieb zu nehmen.

Installationsanleitung

1. Raspberry Pi OS installieren

Falls noch nicht geschehen, installiere Raspberry Pi OS (PiOS) auf einer microSD-Karte und richte den Raspberry Pi 4 entsprechend ein.

2. Repository klonen

Das Code-Repository für den Labyrinth-Roboter muss geklont werden:

- `sudo apt install git python3-dev python3-pip`
- `sudo git clone https://github.com/BenjaminHage/Labyrinth-Maus.git`
- `cd Labyrinth-Maus`

3. Notwendige Python-Bibliotheken installieren

Die folgenden Bibliotheken müssen installiert werden:

- `sudo pip3 install matplotlib --break-system-packages`
- `sudo pip3 install keyboard --break-system-packages`
- `sudo pip3 install scipy --break-system-packages`
- `sudo pip3 install filterpy --break-system-packages`
- `sudo pip3 install numpy --break-system-packages`

4. I2C-Schnittstelle aktivieren und testen

Einige der verwendeten Hardwarekomponenten kommunizieren über das I2C-Protokoll. Daher muss die I2C-Schnittstelle auf dem Raspberry Pi aktiviert werden:

- `sudo pip3 install smbus2 --break-system-packages`
- `sudo raspi-config nonint do_i2c 0`
- `i2cdetect -y 1` #Dient zur Überprüfung.

Der letzte Befehl (`i2cdetect -y 1`) überprüft, ob angeschlossene I2C-Geräte erkannt werden. Wenn eine Tabelle mit Adressen erscheint, ist die I2C-Schnittstelle aktiv.

5. ABElectronics-Python-Bibliotheken installieren

Für den ADC wird die ABElectronics_Python_Libraries-Bibliothek benötigt:

- `sudo git clone https://github.com/abelectronicsuk/ABElectronics_Python_Libraries.git`
- `cd ABElectronics_Python_Libraries`
- `sudo python3 setup.py install`
- `cd ../`

6. Motoron-Bibliothek installieren

Der verwendete Motorcontroller benötigt die motoron-python-Bibliothek von Pololu:

- `sudo git clone https://github.com/pololu/motoron-python.git`
- `cd motoron-python`
- `sudo python3 setup.py install`
- `cd ../`

7. GPIO-Bibliothek für Raspberry Pi installieren

Da die Standard-RPi.GPIO-Bibliothek nicht mehr unterstützt wird, muss stattdessen python3-rpi-lgpio installiert werden:

- `sudo apt remove python3-rpi.gpio`
- `sudo apt update`
- `sudo apt install python3-rpi-lgpio`

8. Adafruit IMU-Treiber installieren

Falls ein ICM20x-IMU-Sensor von Adafruit verwendet wird, muss die passende Bibliothek installiert werden:

- `sudo pip3 install adafruit-circuitpython-icm20x --break-system-packages`

Hardware

Der Roboter

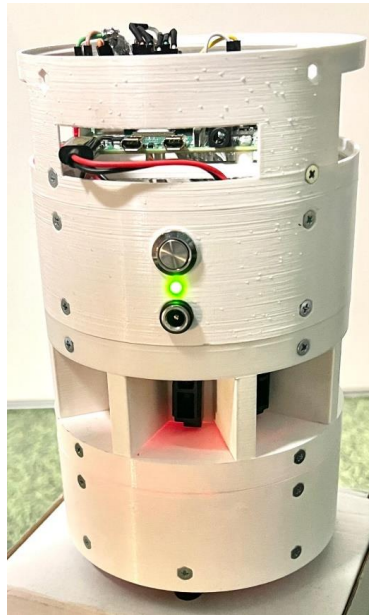


Abbildung 1 Der Roboter

Der Roboter basiert auf einem modularen Aufbau, wobei jedes Modul eine spezifische Funktion übernimmt. Die Module sind in zylindrischen Gehäusen untergebracht mit einem Durchmesser von 11,4 cm und einer Höhe von 4,2 cm. Im Folgenden sind die Module und ihre Komponenten aufgeführt:

Antriebsmodul

- Motoren:
Zwei 6 V DC-Motoren mit 30:1 Getriebe, jeweils ausgestattet mit einem Encoder zur Geschwindigkeitsmessung.
- Räder:
Zwei Kunststoffräder (40 mm Durchmesser, 7 mm Breite).
- Stabilisierung:
Zwei Ballcaster sorgen für eine Stabilisierung des Roboters nach vorne und hinten.
- Motorsteuerung:
Zwei Motoron M1T256 Single I²C Motor Controller, welche über den I²C-Bus angesteuert werden.

Weiterführende Informationen:

- [Motoron Userguide](#)
- [Motoron Python-Dokumentation](#)

Sensormodul

- IR-Sensoren:
Fünf GP2Y0A51SK0F Infrarot-Distanzsensoren zur Wahrnehmung der Labyrinthwände, angeordnet wie folgt:
 - Vorne: 0°
 - Links: 90°
 - Vorne Links: 45°
 - Vorne Rechts: 45°
 - Rechts: 90°

Weiterführende Informationen:

- [IR-Sensoren Datasheet](#)
- IMU:
Ein Adafruit ICM20X Inertial Measurement Unit (IMU) ist zur Messung der Drehgeschwindigkeit integriert. Wird über den I²C-Bus angesteuert.

Weiterführende Informationen:

- [Adafruit ICM20X Library Dokumentation](#)

Steuerungsmodul

- Raspberry Pi 4:
Zentraler Prozessor, der die Steuerungsalgorithmen ausführt und als Hauptrechner fungiert.
- ADC:
Ein ABElectronics ADC Differential Pi, der auf dem Raspberry Pi installiert ist, wandelt die analogen Signale der IR-Sensoren in digitale Werte um. Wird über den I²C-Bus angesteuert.

Weiterführende Informationen:

- [Produktbeschreibung ADC Differential Pi](#)
- [AB Electronics UK ADC Differential Pi Python Library](#)

Akkumodul

- UPS-Modul:
Ein 3S UPS-Modul von Waveshare versorgt den Roboter mit Spannung.
Hat eine I²C-Bus Schnittstelle, über die Informationen ausgelesen werden könnten.

Weiterführende Informationen:

- [Produktbeschreibung UPS Module 3S](#)
- [Beispielcode UPS Module 3S](#)

Das Labyrinth

Das Labyrinth besteht aus mehrfach angeordneten Einheiten, die jeweils ein Quadrat von 36 cm x 36 cm darstellen. Die Anzahl der Einheitsquadrate ist beliebig. Die Wände des Labyrinths sind 10 cm hoch und 1,2 cm dick. Die Außenwand umschließt das gesamte Labyrinth bis auf ein Wandglied, welches als gesuchter Ausgang dient.

Das Labyrinth ist aus 7 3D-gedruckten Elementen gefertigt: einem Wandelement, einem Pfosten-Element und fünf Verbindungsstücken, welche dazu dienen, Pfosten und Wandelemente miteinander zu verbinden.



Abbildung 2 Labyrinth Elemente

Ein Wandglied mit Pfosten stellt 18 cm dar, damit theoretisch auch kleinere Labyrinth aufgebaut werden können. Für dieses Projekt wird als Einheitslänge jedoch zwei gerade verbundene Wandstücke angenommen, welche 36 cm darstellen.

Der Start des Labyrinths kann beliebig gewählt werden, ebenso ist die Orientierung des Roboters beliebig. Das Ziel ist erreicht, wenn der Roboter den Ausgang durchquert hat, dabei muss der Roboter jedoch nicht erkennen, dass er dies gemacht hat.

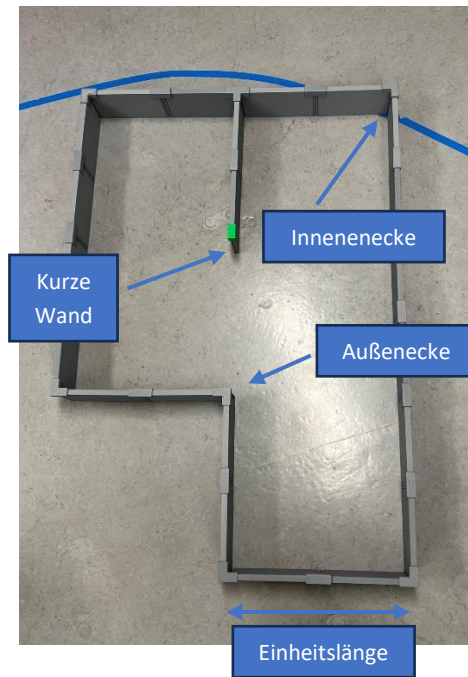


Abbildung 3 Labyrinth Aufbau

Begrifflichkeiten

- **Innenecke:**
Eine Innenecke entsteht, wenn zwei Wände im Labyrinth einen Winkel kleiner als 180° bilden, sodass eine Ecke nach innen zeigt.
- **Außenecke:**
Eine Außenecke ist das Gegenteil einer Innenecke, also ein Punkt, an dem zwei Wände einen Winkel größer als 180° bilden, sodass die Ecke nach außen zeigt. Diese Art von Ecke ist schwieriger zu befahren, da der Roboter sich zeitweise nicht an einer Wand orientieren kann, weil sie nicht von den Sensoren wahrgenommen wird.
- **Kurze Wand:**
Eine kurze Wand ist eine Wand, die so klein ist, dass sie nicht gleichzeitig von den seitlichen und diagonalen Sensoren des Roboters erfasst werden kann. Sie kann sich also in einem toten Winkel für die Sensoren befinden, sodass der Roboter sie während der Bewegung nicht kontinuierlich erkennt. Im Labyrinth entspricht, das der Seite eines Pfostens an der keine wand anschließt.

Programmnutzung

Starten des Programms

Das Programm kann über die Konsole gestartet werden, zum Beispiel direkt im Betriebssystem des Raspberry Pis oder, wie empfohlen, über eine SSH-Verbindung mit dem Pi.

Bevor das Programm gestartet wird, muss sichergestellt werden, dass der pigpio-Daemon läuft. Dieser wird mit folgendem Befehl gestartet:

- `sudo pigpiod`

Um das Hauptskript auszuführen, navigiere in das geklonte Repository und starte `maus.py`:

- `cd Labyrinth-Maus`
- `python3 maus.py [OPTIONEN]`

Das Programm kann jederzeit mit Strg + C beendet werden.

Verfügbare Befehlszeilenparameter

Das Skript `maus.py` akzeptiert folgende Parameter zur Steuerung des Roboters:

Parameter	Langform	Beschreibung
-a	--autonomous	Aktiviert den automatischen Modus, in dem der Roboter eigenständig das Labyrinth durchquert.
-f	--filename	Gibt eine alternative Datei mit Parametern für die Sensoren an, die für die Umrechnung der Spannung in eine Distanz verwendet werden.

Ein Beispielaufruf für den automatischen Modus könnte folgendermaßen aussehen:

- `python3 maus.py -a`

Manuelle Steuerung

Falls der manuelle Modus genutzt wird (ohne -a), kann der Roboter über die Tastatur gesteuert werden:

- Pfeil oben: Vorwärts
- Pfeil unten: Rückwärts
- Pfeil links: Nach links drehen
- Pfeil rechts: Nach rechts drehen

- A: Wechselt zwischen manuellem und autonomem Modus
- C: Beendet das Programm

Automatikmodus

Im automatischen Modus (-a) navigiert der Roboter selbstständig durch das Labyrinth, wobei er seine Sensoren zur Umgebungserfassung nutzt.

Es ist möglich, das Programm manuell zu beenden oder in den manuellen Modus zu wechseln. Auch per Konsole und durch Drücken von Strg + C kann das Programm beendet werden.

Fehlerbehebung

Falls der Roboter nicht wie erwartet funktioniert, können folgende Maßnahmen helfen:

- Überprüfen, ob pigpio läuft:
 - `sudo systemctl status pigpiod`
- Falls das Parameter-File nicht gefunden wird:
 - Stelle sicher, dass die Datei existiert und der richtige Pfad angegeben wurde.
 - Beispiel für einen manuellen Aufruf mit Parameter-File:
 - `python3 maus.py -f pfad/zur/datei`
- Bei Problemen mit dem I2C-Bus:
 - Überprüfe, ob alle I2C Adressen erkannt werden und mit den in den Dateien Hinterlegten Adressen übereinstimmen.
 - `i2cdetect -y 1`

Konsolenausgabe

Dieser Abschnitt beschreibt die Konsolenausgaben des Programms, die während des Betriebs des Roboters angezeigt werden. Diese Ausgaben liefern wichtige Informationen zur aktuellen Steuerung und zum Zustand des Roboters.

1. Allgemeine Struktur der Konsolenausgabe

Die Konsolenausgabe wird in mehreren Zeilen strukturiert dargestellt und zeigt sowohl die aktuellen Sensordaten als auch die Steuerparameter des Roboters.

Ein Beispiel für eine typische Konsolenausgabe im manuellen Modus:

```
-----
Left Wheel Velocity:      0.00 m/s
Left Wheel Velocity target: 0.00 m/s
P:  0.00    I:  0.00    D:  0.00    PID:  0.00

Right Wheel Velocity:      0.00 m/s
Right Wheel Velocity target: 0.00 m/s
P:  0.00    I:  0.00    D:  0.00    PID:  0.00

Base_Speed:                0.00 m/s
Angle:                     -3.30 °
Angle_setpoint:             -3.30 °
Angle_control:              0.02
Sensor Readings:           ['13.3142', '18.4812', '11.0316', '10.7170', '10.3102']
```

Abbildung 4Konsolenausgabe manueller Modus

2. Bedeutung der einzelnen Werte

- Left Wheel Velocity / Right Wheel Velocity: Die aktuelle Geschwindigkeit der linken bzw. rechten Antriebsräder in m/s.
- Left Wheel Velocity target / Right Wheel Velocity target: Die angestrebte Geschwindigkeit der Räder.
- PID-Werte (P, I, D, PID): Die einzelnen Komponenten des PID-Reglers, die zur Steuerung der Radgeschwindigkeit beitragen.
- Base_Speed: Die Basisgeschwindigkeit des Roboters.
- Angle : Der aktuelle Winkel und der gewünschte Winkel in Grad.
- Angle_setpoint: Wurde verwendet als versucht wurde den Roboter mit einer Winkel Vorgabe zu steuern, gibt derzeit nur den aktuellen Winkel an.
- Angle_control: Wurde verwendet als versucht wurde den Roboter mit einer Winkel Vorgabe zu steuern. der Wert des Reglers wird derzeit nicht verwendet.

- Sensor Readings: Die aktuellen Messwerte der Abstandssensoren. Dabei sind die Sensorindexe folgendermaßen:
 - 0 = Vorne
 - 1 = Vorne Links
 - 2 = Links
 - 3 = Vorne Rechts
 - 4 = Rechts

3. Automatikmodus

Im automatischen Modus werden zusätzliche Informationen zur Entscheidungsfindung des Roboters ausgegeben:

```
-----
State:          6
follow_sensor:  0
pledge_count:   0

front_sensor:   0.3053
front_left_sensor: 0.2352   False
left_sensor:    0.6039   False
front_right_sensor: 0.4898   False
right_sensor:   0.1545   True

Angle:          270.16 °
Angle_setpoint: 271.09 °
x: 0.0675 y:-0.2225   target_x: 0.0000 target_y: 0.0000

detekt edge, start driving forward
```

Abbildung 5 Konsolenausgabe autonomer Modus

Zusätzliche Werte im Automatikmodus:

- State: Der aktuelle Zustand des Zustandsautomaten
- follow_sensor: Variable innerhalb der Steuerlogic. Gibt an welcher Sensor zum Folgen der Wand genutzt wird.
- pledge_count: Zähler für den Pledge-Algorithmus.
- Sensormesswerte: Jeder Sensor zeigt den aktuellen Wert und ob er als aktiv angesehen wird.
- x, y, target_x, target_y: Position des Roboters und die Werte mit denen die target Variablen gesetzt sind.
- Steuerennachricht: Eine Beschreibung der nächsten Aktion des Roboters.

Die Konsolenausgaben werden auch in eine Datei (info_lines_log.log) gespeichert jedesmal wenn sich die Steuernachricht ändert, um sie später analysieren zu können.

Dokumentation des Zustandsautomaten und des Pledge-Algorithmus

Der Pledge-Algorithmus ist ein Verfahren zur Navigation in unbekannten, labyrinthartigen Umgebungen. Er ermöglicht es einem Roboter, Hindernisse zu umgehen und dabei seine ursprüngliche Bewegungsrichtung beizubehalten. Dieser Abschnitt beschreibt die Implementierung des Algorithmus als Zustandsautomat.

Der Pledge-Algorithmus basiert auf folgenden Schritten:

- Bewegung in einer definierten Richtung: Der Roboter bewegt sich in einer festgelegten Richtung, bis er auf eine Wand trifft.
- Folgen der Wand: Beim Auftreffen auf eine Wand beginnt der Roboter, der Wand an einer Seite (z. B. links oder rechts) zu folgen.
- Drehungszählung: Während des Folgens der Wand zählt der Roboter jede Drehung:
 - Linksdrehungen: +1
 - Rechtsdrehungen: -1

Verlassen der Wand: Sobald die Summe der Drehungen wieder null ist, verlässt der Roboter die Wand und setzt seine ursprüngliche Bewegungsrichtung fort.

Der Algorithmus garantiert, dass der Roboter einen Ausgang findet, sofern einer existiert.

Zustandsautomat

Der Zustandsautomat ist das Herzstück der Implementierung. Er besteht aus 22 Zuständen, die das Verhalten des Roboters in verschiedenen Situationen steuern. Jeder Zustand repräsentiert eine spezifische Aktion oder Regelung, und die Übergänge zwischen den Zuständen werden durch Sensordaten und interne Logik gesteuert.

Der grundsätzliche Ablauf ist wie folgt

Initialisierungsphase

Erste Prüfung – Wird eine Wand erkannt?

- Falls ja:
 - Drehe den Roboter so, dass der Frontsensor auf den Punkt zeigt, an dem der Sensor den geringsten Abstand zur Wand gemessen hat.
- Falls nein:
 - Beginne mit einer langsamen Drehung.

- Wird eine Wand entdeckt (noch bevor sich der Roboter einmal vollständig im Kreis gedreht hat), dann dreht er sich sofort so, dass der Frontsensor auf die Wand ausgerichtet ist.
- Falls während der Drehung keine Wand erkannt wurde, fährt der Roboter geradeaus, bis eine Wand erkannt wird.

Orthogonale Ausrichtung:

- Nachdem der Roboter die Wand mit dem Frontsensor fixiert hat, dreht er sich einmal im Kreis.
- Dabei merkt er sich den Winkel, bei dem der gemessene Abstand zur Wand am geringsten war.
- Anschließend dreht er sich in diese Richtung.

Wandabstand:

- Mindestens ein diagonaler Sensor erkennt eine Wand:
 - Der Roboter steht nun orthogonal zur Wand und fährt zum definierten Sollabstand.
- Keiner der diagonalen Sensoren registriert eine Wand:
 - Es wird angenommen, dass der Roboter frontal auf eine Außenecke blickt. In diesem Fall hält er einen etwas größeren Abstand als der Sollabstand ein.

Folge Richtung:

- Beide diagonale Sensoren erkennen eine Wand:
 - Die Ausrichtung spielt hier keine große Rolle; der Roboter dreht sich standardmäßig nach links.
- Nur ein diagonaler Sensor erkennt eine Wand:
 - Der Roboter befindet sich an einer Außenecke und dreht sich so, dass er sich von der Ecke entfernt, wenn er der Wand folgt.
- Keiner der diagonalen Sensoren erkennt eine Wand:
 - Der Roboter dreht sich um 45° nach links und fährt geradeaus, bis er über den rechten Sensor wieder eine Wand erfasst.

Damit ist die Initialisierungsphase abgeschlossen.

Wandfolge

Seitliche Abstandskontrolle:

- Der Roboter hält kontinuierlich den Abstand zur Wand mithilfe des seitlichen Sensors.

Umgang mit Innenecken:

- Erkennung:
 - Kommt der Roboter in den Bereich einer Innenecke, signalisiert der Frontsensor dies.
- Vorgehen:
 - Sobald er nah genug an der Wand ist, regelt er den Abstand zur vorderen Wand auf den Sollabstand.
 - Anschließend dreht er sich **90°** (links oder rechts – abhängig davon, ob er der rechten oder linken Wand folgt) und setzt die Geradeausfahrt fort, während er weiterhin den seitlichen Abstand kontrolliert.

Umgang mit Außenecken:

- Erkennung:
 - Außenecken werden über die diagonalen Sensoren festgestellt.
- Fall: Pledge-Zähler $\neq 0$
 - Außenecke erkennen und Freilösen:
 - Der Roboter erkennt die Außenecke und löst sich von der bisherigen Wandführung.
 - Geradeausfahrt bis zur Kante:
 - Er fährt geradeaus, bis er die Kante (den tatsächlichen Eckpunkt) erreicht.
- Zielpunkt setzen:
 - Sobald die Kante erreicht ist, wird ein Zielpunkt definiert, der auf der Winkelhalbierenden der Außenecke liegt und direkt vor dem Roboter positioniert ist.
- Fahrt zum Zielpunkt:

- Der Roboter fährt geradeaus auf diesen Zielpunkt zu und regelt dabei seinen Abstand, wenn er dem Punkt nahe ist.
- Drehung:
 - Nach Erreichen des Zielpunkts dreht sich der Roboter um 90°, je nach je nach gefolgter wand seite
- Fahrt zur Kante auf neuer Seite:
 - Nach der Drehung fährt der Roboter zunächst geradeaus.
 - Dabei wird die Wand in der Regel zuerst über den diagonalen Sensor erkannt. Sobald der diagonale Sensor signalisiert, dass der Roboter der Wand zu nahekommt, wird die diagonale Regelung aktiviert. Ansonsten fährt er weiter gerade aus
 - Sobald der seitliche Sensor die Wand erkennt, wechselt der Roboter in die normalen Wandfolge.
 - Spezialfall: Kurze Wand:
 - Bei diagonaler Wandführung:

Löst der diagonale Sensor eine negative Flanke aus, fährt der Roboter geradeaus, bis der rechte Sensor die Wand erfasst.
 - Falls der seitliche Sensor die Wand erfasst, während kein diagonalen Sensor sie registriert:

löst der Roboter sich wieder von der aktuellen Wand und fährt die neue Außenecke an.
- i.
- Fall: Pledge-Zähler = 0
 - In diesem Fall setzt der Roboter keinen Zielpunkt und fährt geradeaus, bis er eine Wand vor sich erkennt.
 - Daraufhin regelt er den Abstand zu wand bis er den Sollabstand erreicht

Fehlererkennung und Reinitialisierung:

Sollte der Roboter unerwartet plötzlich eine Wand sehr nahe vor sich feststellen, sei es während der Geradeausfahrt an einer Außenecke oder beim diagonalen Folgen, wird dies als Fehler interpretiert, und der Roboter reinitialisiert seine Initialisierungsphase.

Zustände und Zustandsübergänge

Zustand 0: Initialisierung

Beschreibung:

Der Roboter überprüft die Sensordaten und entscheidet, ob eine Wand erkannt wurde. Dieser Zustand dient als Startpunkt für die Navigation.

Aktionen:

- Setzt follow_sensor auf eine leere Liste.
- Setzt pledge_count auf eine leere Liste.
- Setzt die Geschwindigkeiten der linken und rechten Räder auf 0.
- Setzt on_point auf False.

Übergänge:

- Zustand 3:
Eine Wand auf der rechten Seite wurde erkannt. Der Roboter bereitet sich darauf vor, eine 90°-Drehung nach rechts durchzuführen.
(Steuernachricht: "detect wall at the right, set up turning to it")
- Zustand 4:
Eine Wand auf der linken Seite wurde erkannt. Der Roboter bereitet sich darauf vor, eine 90°-Drehung nach links durchzuführen.
(Steuernachricht: "detect wall at the left, set up turning to it")
- Zustand 11:
Eine Wand vorne links wurde erkannt. Der Roboter bereitet sich darauf vor, eine 45°-Drehung nach links durchzuführen.
(Steuernachricht: "detect wall at the front_left, set up turning to it")
- Zustand 10:
Eine Wand vorne rechts wurde erkannt. Der Roboter bereitet sich darauf vor, eine 45°-Drehung nach rechts durchzuführen.
(Steuernachricht: "detect wall at the front_right, set up turning to it")
- Zustand 12:
Eine Wand direkt vorne wurde erkannt. Der Roboter bereitet sich darauf vor, eine 0°-Drehung (geradeaus ausrichten) durchzuführen.
(Steuernachricht: "detect wall at the front, set up turning to it")

- Zustand 15: Keine Wand wurde erkannt. Der Roboter bereitet sich darauf vor, eine 360°-Drehung durchzuführen, um die Umgebung zu scannen.
(Steuernachricht: "no wall detected, set up 360° turn")

Zustand 1: Linker Wand folgen

Beschreibung:

Der Roboter folgt einer linken Wand und hält dabei einen definierten Abstand ein.

Aktionen:

- Regelt den Abstand zur linken Wand mithilfe eines PID-Reglers. Dabei passt er die Geschwindigkeiten der Räder an, um den Abstand zur Wand zu halten.
- Setzt follow_sensor auf self.left.

Übergänge:

- Zustand 6:
Der Roboter erkennt eine Kante (keine Wand mehr auf der linken Seite). Er beginnt, gradeaus zu fahren. (Steuernachricht: "detect edge, start driving forward")
- Zustand 9:
Der Roboter erkennt eine Wand vorne innerhalb des Regelabstands. Er beginnt, den Abstand zur vorderen Wand zu regeln.
(Steuernachricht: "found front wall, start controlling the distance to it")

Zustand 2: Rechter Wand folgen

Beschreibung:

Der Roboter folgt einer rechten Wand und hält dabei einen definierten Abstand ein.

Aktionen:

- Regelt den Abstand zur rechten Wand mithilfe eines PID-Reglers. Dabei passt er die Geschwindigkeiten der Räder an, um den Abstand zur Wand zu halten.
- Setzt follow_sensor auf self.right.

Übergänge:

- Zustand 6:
Der Roboter erkennt eine Kante (keine Wand mehr auf der rechten Seite). Er beginnt, gradeaus zu fahren.
(Steuernachricht: "detect edge, start driving forward")
- Zustand 9:
Der Roboter erkennt eine Wand vorne innerhalb des Regelabstands. Er beginnt, den

Abstand zur vorderen Wand zu regeln.

(Steuernachricht: "found front wall, start controlling the distance to it")

Zustand 3: Vorbereitung auf 90°-Drehung nach rechts

Beschreibung:

Der Roboter bereitet eine 90°-Drehung nach rechts vor, um sich parallel zu einer rechten Wand auszurichten.

Aktionen:

- Berechnet den Zielwinkel (angle_setpoint) basierend auf dem aktuellen Winkel (theta) und dem vorherigen Zustand.
- Setzt follow_sensor auf self.left, wenn der Roboter zuvor einer vorderen Wand gefolgt ist.
- Verringert den pledge_count um 1, falls dieser nicht leer ist.
- Setzt die Geschwindigkeiten der Räder auf 0.

Übergang:

- Zustand 5:
Die Drehung wird gestartet. (Steuernachricht: "set up 90° right turn, start turning")

Zustand 4: Vorbereitung auf 90°-Drehung nach links

Beschreibung:

Der Roboter bereitet eine 90°-Drehung nach links vor, um sich parallel zu einer linken Wand auszurichten.

Aktionen:

- Berechnet den Zielwinkel (angle_setpoint) basierend auf dem aktuellen Winkel (theta) und dem vorherigen Zustand.
- Setzt follow_sensor auf self.right, wenn der Roboter zuvor einer vorderen Wand gefolgt ist.
- Erhöht den pledge_count um 1, falls dieser nicht leer ist.
- Setzt die Geschwindigkeiten der Räder auf 0.

Übergang:

- Zustand 5:
Die Drehung wird gestartet.
(Steuernachricht: "set up 90° left turn, start turning")

Zustand 5: Geregelte Drehung

Beschreibung:

Der Roboter führt eine geregelte Drehung durch, um einen vorgegebenen Sollwinkel zu erreichen.

Aktionen:

- Regelt die Drehung mithilfe eines PID-Reglers und einer linearen Rückführung der Drehgeschwindigkeit, um den Sollwinkel (angle_setpoint) zu erreichen.
- Passt die Geschwindigkeiten der Räder an, um die Drehung zu steuern.

Der PID-Regler berechnet die notwendige Korrektur basierend auf dem Fehler zwischen Soll- und Ist-Winkel. Die lineare Rückführung gleicht die Differenz zwischen der Basisdrehgeschwindigkeit und der aktuellen Drehgeschwindigkeit aus. Addiert ergeben die beiden Anteile die Radgeschwindigkeitsvorgaben. Dies hat den Hintergrund, dass bei kleiner initialer Differenz zwischen Soll- und Ist-Winkel der Roboter nicht zu drehen begann und ein stärkeres Wählen der Parameter ein zu großes Überschwingverhalten verursachte. Besonders das I-Glied war in dieser Hinsicht schwer zu handhaben, da es bei großen Winkeln das Überschwingen besonders verstärkte. Bei kleinen Winkeln hingegen erwies es sich jedoch als sehr hilfreich, um das Starten des Roboters zu gewährleisten und sicherzustellen, dass es zu keiner statischen Regelabweichung kam, weil der Roboter vorzeitig stehen blieb. Die lineare Rückführung der Drehgeschwindigkeit hat diese Probleme nahezu vollständig gelöst. Da deren Anteil im Stand dafür sorgt, dass der Roboter zu drehen begann, unabhängig von der Abweichung des Winkels, und auch dafür sorgte, dass der Roboter nicht zu sehr an Geschwindigkeit verliert um den Sollwinkel herum, sodass er vorzeitig stehen bleibt. Dazu ergab sich zusätzlich der Vorteil, dass dies zu einer gleichmäßigeren Drehbewegung führt und man das I-Glied nicht mehr benötigte.

Übergänge:

- Zustand 15:
Der Roboter hat sich in der Initialisierungsphase so gedreht, dass der vordere Sensor in Richtung einer erkannten Wand zeigt und möchte sich jetzt orthogonal zu dieser ausrichten.
(Steuernachricht: "turned to front wall, start ESC")
- Zustand 19:
Der Roboter hat sich parallel zu einer rechten Wand ausgerichtet. Er bereitet die PID-Regelung für die rechte Wand vor.
(Steuernachricht: "turned parallel to right wall, set up PID for right wall")
- Zustand 18:
Der Roboter hat sich parallel zu einer linken Wand ausgerichtet. Er bereitet die PID-Regelung für die linke Wand vor.

(Steuernachricht: "turned parallel to left wall, set up PID for left wall")

- Zustand 6:
Falls der Roboter in der Initialisierungsphase sich 360° gedreht hat um nach einer Wand zu suchen.
(Steuernachricht: "did turned, start search by driving forward")
- Zustand 6:
Falls nach einer orthogonalen Drehung ist der Abstand zur vorderen Wand groß ist, beginnt der Roboter, geradeaus zu fahren.
(Steuernachricht: "did orthogonal turn, start driving forward")
- Zustand 9:
Nach einer orthogonalen Ausrichtung zur vorderen Wand, falls sich die Wand im Regelabstand befindet. Der Roboter beginnt, den Abstand zur vorderen Wand zu regeln.
(Steuernachricht: "did orthogonal turn, start controlling to near front wall")
- Zustand 6:
Nach dem der Roboter sich auf einem Zielpunkt oder in der Initialisierungsphase um 45° gedreht hat
(Steuernachricht: "did on point turn, start driving forward")

Zustand 6: Geradeausfahren

Beschreibung:

Der Roboter fährt geradeaus, ohne eine spezifische Regelung.

Aktionen:

- Setzt die Radgeschwindigkeit auf die Basisgeschwindigkeit.
- Ein zusätzlicher Anteil eines PID-Reglers auf Radgeschwindigkeit sorgt dafür, dass der Winkel des Roboters konstant bleibt.
- Wenn der Roboter zuvor einen Zielpunkt gesetzt hat (`prev_state == 7`), wird stattdessen die Bewegung zum Zielpunkt geregelt, sodass der Roboter auf den Zielpunkt zusteuert.
- Setzt `on_point` auf False.
- Setzt `follow_sensor` auf `self.front`, wenn der `pledge_count` 0 ist.

Übergänge:

- Zustand 7:
Der Roboter erkennt, dass er über eine Kante gefahren ist und setzt einen Zielpunkt, falls der `pledge_count` nicht 0 ist.

(Steuernachricht: "over edge, set target point")

- Zustand 8:
Der Roboter nähert sich einem Zielpunkt und beginnt, die Bewegung zum Punkt zu regeln.
(Steuernachricht: "near target point, start controlling to it")
- Zustand 21:
Der Roboter erkennt eine Wand vorne links und beginnt, den Abstand zu dieser Wand zu regeln, falls der Roboter sich dieser zu sehr annähert.
(Steuernachricht: "arrived at front-left wall, start controlling to it")
- Zustand 22:
Der Roboter erkennt eine Wand vorne rechts und beginnt, den Abstand zu dieser Wand zu regeln, falls der Roboter sich dieser zu sehr annähert.
(Steuernachricht: "arrived at front-right wall, start controlling to it")
- Zustand 13:
Der Roboter erkennt eine vordere Wand bei einem Pledge-Zähler von 0.
(Steuernachricht: "ausrichtung gerade wand vorne für pledge wechsel aber erstmal übersprungen")

Hier hätte man implementieren können, dass der Roboter sich wieder neu orthogonal zu einer vorderen Wand ausrichten, nach dem er sich von der zuvor gefolgten Wand bei einer Außenecke gelöst hat weil der Pledge – Zähler 0 ist. Dieser Schritt wurde aber verworfen bzw. übersprungen.

- Zustand 18:
Der Roboter erkennt eine linke Wand nach einer linken Außenecke und bereitet die PID-Regelung für die linke Wand vor.
(Steuernachricht: "arrived at left wall, setup PID for left wall")
- Zustand 19:
Der Roboter erkennt eine rechte Wand nach einer rechten Außenecke und bereitet die PID-Regelung für die rechte Wand vor.
(Steuernachricht: "arrived at right wall, setup PID for right wall")
- Zustand 0:
Der Roboter erkennt eine Wand in der Initialisierungsphase und startet den Initialisierungsprozess erneut.

(Steuernachricht: "found a wall, restart init")

- Zustand 9:
Der Roboter erkennt eine vordere Wand innerhalb des Regelabstands und beginnt, den Abstand zu regeln.
(Steuernachricht: "front wall is near, start to control the distance")
- Zustand 0:
Der Roboter erkennt eine unerwartete vordere Wand und startet den Initialisierungsprozess erneut.
(Steuernachricht: "error unexpected front wall, restart init")

Zustand 7: Zielpunkt setzen

Beschreibung:

Der Roboter setzt einen Zielpunkt, zu dem er sich bewegen soll.

Aktionen:

- Berechnet den Zielpunkt (target_x, target_y) basierend auf der aktuellen Position und Ausrichtung des Roboters.

Übergang:

- Zustand 6: Der Roboter beginnt, sich zum Zielpunkt zu bewegen. (Steuernachricht: "set up forward point, start driving forward to it")

Zustand 8: Regelung zum Zielpunkt

Beschreibung:

Der Roboter regelt die Bewegung, um einen bestimmten Zielpunkt zu erreichen.

Aktionen:

- Ein PID-Regler ermittelt seinen Wert basierend auf dem aktuellen Abstand zum Zielpunkt.
- Ein weiterer PID-Regler ermittelt seinen Wert basierend auf dem Unterschied zwischen dem Winkel des Roboters und dem relativen Winkel zum Zielpunkt.
- Setzt die Radgeschwindigkeiten basierend auf den Reglern.
- Setzt on_point auf True.

Übergänge:

- Zustand 4:
Der Roboter hat den Zielpunkt erreicht und bereitet eine 90°-Drehung nach links vor.

(Steuernachricht: "got to point, start set up left turn")

- Zustand 3:
Der Roboter hat den Zielpunkt erreicht und bereitet eine 90°-Drehung nach rechts vor.
(Steuernachricht: "got to point, start set up right turn")

Zustand 9: Regelung zur vorderen Wand

Beschreibung:

Der Roboter regelt den Abstand zu einer vorderen Wand.

Aktionen:

- Falls keine Wand vorne links oder vorne rechts erkannt wird, wird der `front_desired_distance_faktor` gesetzt, welcher den Sollabstand modelliert.
- Regelt den Abstand zur vorderen Wand mithilfe eines PID-Reglers.

Wenn weder vorne links oder vorne rechts eine Wand erkannt wird obwohl direkt vor ihm eine registriert ist, blickt der Roboter frontal auf eine Außenecke. In diesen Fall hat es sich bewährt einen größeren Abstand als der normale Sollabstand zu halten. Denn so wird es vermieden dass der Roboter, wenn er die Ecke passieren möchte, an diese anstößt.

Übergänge:

- Zustand 3:
Der Roboter hat den gewünschten Abstand zur vorderen Wand erreicht, während er einer linken Wand folgt und bereitet eine 90°-Drehung nach rechts vor.
(Steuernachricht: "got to wall distance, start set up right turn")
- Zustand 4:
Der Roboter hat den gewünschten Abstand zur vorderen Wand erreicht, während er einer rechten Wand folgt und bereitet eine 90°-Drehung nach links vor.
(Steuernachricht: "got to wall distance, start set up left turn")
- Zustand 4:
Falls in der Initialisierungsphase der Roboter orthogonal frontal ausgerichtet ist und der Frontsensor links aktiv ist.
(Steuernachricht: "got to wall distance, detect wall front left, start set up left turn")
- Zustand 3:
Falls in der Initialisierungsphase der Roboter orthogonal frontal ausgerichtet ist und

der Sensor vorne rechts aktiv ist und nicht vorne links.

(Steuernachricht: "got to wall distance, detect wall front right, start set up right turn")

- Zustand 11:

Falls in der Initialisierungsphase der Roboter orthogonal frontal ausgerichtet ist, jedoch weder ein aktiver Sensor vorne links noch vorne rechts vorliegt.

(Steuernachricht: "got to wall distance, no front left, front right wall, start set up light left turn")

Wenn in der Initialisierungsphase beide vorderen seitlichen Sensoren aktiv sind, der Roboter also normal auf eine Wand schaut, entscheidet es sich links zu drehen.

Befindet sich nur einer der beiden Sensoren in Aktivierung, schaut er auf eine Wand nahe einer Außenecke, wobei der Roboter sich so ausrichtet, dass er der Wand folgt, die der aktiven Seite zugeordnet ist, und somit von der Ecke weggeführt. Ist keiner der seitlichen Sensoren aktiv, ist der Roboter direkt auf die Außenecke ausgerichtet und führt eine 45°-Drehung durch. Steht er exakt auf der Winkelhalbierenden, erreicht er die Wand parallel. Kleinere Abweichungen werden durch die Regler kompensiert.

Zustand 10: Vorbereitung auf 45°-Drehung nach rechts

Beschreibung:

Der Roboter bereitet eine 45°-Drehung nach rechts vor.

Aktionen:

- Setzt den Zielwinkel (angle_setpoint) auf den aktuellen Winkel (theta) minus 45°.
- Setzt die Geschwindigkeiten der Räder auf 0.

Übergang:

- Zustand 5:
Die Drehung wird gestartet.
(Steuernachricht: "set up 45° right turn, start turning")

Zustand 11: Vorbereitung auf 45°-Drehung nach links

Beschreibung:

Der Roboter bereitet eine 45°-Drehung nach links vor.

Aktionen:

- Setzt den Zielwinkel (angle_setpoint) auf den aktuellen Winkel (theta) plus 45°.
- Setzt follow_sensor auf self.right, wenn der Roboter zuvor einer vorderen Wand gefolgt ist.
- Setzt die Geschwindigkeiten der Räder auf 0.

Übergang:

- Zustand 5:
Die Drehung wird gestartet.
(Steuernachricht: "set up 45° left turn, start turning")

Zustand 12: Vorbereitung auf 0°-Drehung

Beschreibung:

Der Roboter bereitet eine 0°-Drehung (geradeaus ausrichten) vor.

Aktionen:

- Setzt den Zielwinkel (angle_setpoint) auf den aktuellen Winkel (theta).
- Setzt die Geschwindigkeiten der Räder auf 0.

Übergang:

- Zustand 5:
Die Drehung wird gestartet.
(Steuernachricht: "set up 0° turn, start turning")

Zustand 13: ESC (Extremum Seeking Control)

Beschreibung:

Dieser Zustand war für die Nutzung eines ESC-Reglers zur orthogonalen Ausrichtung zu einer Wand gedacht. Dies wurde jedoch verworfen bzw. wird übersprungen.

Aktionen:

- Setzt die Radgeschwindigkeiten auf die Basisgeschwindigkeit.
- Setzt follow_sensor auf self.front.

Übergang:

- Zustand 6:
Unmittelbar nach Eintritt in den Zustand

Zustand 14: Ungeregelt drehen

Beschreibung:

Der Roboter dreht sich mit konstanter Geschwindigkeit.

Dieser Zustand wird nur zweimal und in der Initialisierungsphase verwendet: Zum einen, um nach einer Wand zu suchen, wenn keine direkt erkannt wurde, oder um den Winkel zu finden, bei dem der Roboter orthogonal zur Wand steht.

Aktionen:

- Setzt die Geschwindigkeiten der Räder auf einen fixen Wert, damit sich der Roboter mit der Basisdrehgeschwindigkeit auf der Stelle dreht.
- Speichert den Winkel mit dem kleinsten Abstand zur Wand (`min_distance_angle`), falls der Roboter dabei ist, sich orthogonal auszurichten.

Übergänge:

- Zustand 0:
Der Roboter erkennt eine Wand und startet den Initialisierungsprozess erneut.
(Steuernachricht: "found a wall, restart init")
- Zustand 20:
Der Roboter hat eine 360°-Drehung abgeschlossen und richtet sich auf den kleinsten Abstand zur Wand aus.
(Steuernachricht: "did 360° turn, set angle_setpoint to smallest front distance")
- Zustand 6:
Der Roboter hat keine Wand gefunden, bis er sich einmal im Kreis gedreht hat, und beginnt, geradeaus zu fahren.
(Steuernachricht: "found no wall after 360° turn, start driving forward")

Zustand 15: Vorbereitung auf 360°-Drehung

Beschreibung:

Der Roboter bereitet eine 360°-Drehung vor, um die Umgebung zu scannen oder sich orthogonal zu einer Wand auszurichten.

Aktionen:

- Setzt den Zielwinkel (`angle_setpoint`) auf den aktuellen Winkel (`theta`) plus 360°.
- Setzt die Geschwindigkeiten der Räder auf 0.
- Setzt `follow_sensor` auf `self.front`, wenn der vorherige Zustand 5 war.

Übergang:

- Zustand 14:
Die Drehung wird gestartet.
(Steuernachricht: "set up 360° turn, start turning")

Zustand 16: (Reserviert)

Beschreibung:

Dieser Zustand ist reserviert und wird derzeit nicht verwendet.

Zustand 17: (Reserviert)

Beschreibung:

Dieser Zustand ist reserviert und wird derzeit nicht verwendet.

Zustand 18: Vorbereitung PID-Regelung für linke Wand

Beschreibung:

Der PID-Regler zum Folgen der linken Wand wird vorbereitet.

Aktionen:

- Initialisiert den PID-Regler für die linke Wand, indem die vorherige Regelabweichung auf die jetzige gesetzt wird.
- Initialisiert den `pledge_count` auf 0, falls dieser leer ist.

Das Setzen der Vorherigen Regelabweichung welche für das Bestimmen des Differenzials für das D-Glied verwenden wird, ist nötig weil bei der Regler bei erstmaliger Verwendung viel zu stark auslagern könnte, aufgrund dessen, dass ein Falschen und viel zu hohes Differenzial berechnet wird.

Übergänge:

- Zustand 1:
Der Roboter beginnt, der linken Wand zu folgen.
(Steuernachricht: "did setup, start following left wall")
- Zustand 6:
Der Roboter erkennt eine Kante (keine Wand mehr vorne links) und beginnt, geradeaus zu fahren.
(Steuernachricht: "detect edge, start driving forward")

Der Übergang zum Geradeausfahren kommt dann vor, wenn der Roboter mit dem Folgen einer Wand beginnen möchte, sich aber bereits bei einer Außenecke befindet. Dies kommt vor allem dann vor, wenn nach einer Außenecke der Roboter an einer dünnen Wand ankommt.

Zustand 19: PID-Regelung für rechte Wand

Beschreibung:

Der PID-Regler zum Folgen der rechten Wand wird vorbereitet.

Aktionen:

- Initialisiert den PID-Regler für die rechte Wand, indem die vorherige Regelabweichung auf die jetzige gesetzt wird.
- Initialisiert den `pledge_count` auf 0, falls dieser leer ist.

Das Setzen der Vorherigen Regelabweichung welche für das Bestimmen des Differenzials für das D-Glied verwenden wird, ist nötig weil bei der Regler bei erstmaliger Verwendung viel zu stark auslagen könnte, aufgrund dessen, dass ein Falschen und viel zu hohes Differenzial berechnet wird.

Übergänge:

- Zustand 2:
Der Roboter beginnt, der rechten Wand zu folgen.
(Steuernachricht: "did setup, start following right wall")
- Zustand 6:
Der Roboter erkennt eine Kante (keine Wand mehr vorne rechts) und beginnt, geradeaus zu fahren.
(Steuernachricht: "detect edge, start driving forward")

Der Übergang zum Geradeausfahren kommt dann vor, wenn der Roboter mit dem Folgen einer Wand beginnen möchte, sich aber bereits bei einer Außenecke befindet. Dies kommt vor allem dann vor, wenn nach einer Außenecke der Roboter an einer dünnen Wand ankommt.

Zustand 20: Winkel mit kleinstem Abstand einstellen

Beschreibung:

Der Roboter stellt den Winkel ein, bei dem der Abstand zur Wand am kleinsten ist.

Aktionen:

- Setzt den Zielwinkel (angle_setpoint) basierend auf dem kleinsten Abstand zur Wand (min_distance_angle).
- Setzt die Geschwindigkeiten der Räder auf 0.

Übergang:

- Zustand 5:
Der Roboter beginnt die geregelte Drehung, um sich auf den kleinsten Abstand zur Wand und somit orthogonal zur Wand auszurichten.
(Steuernachricht: "set smallest distance angle, start turning")

Zustand 21: Links diagonal folgen

Beschreibung:

Der Roboter folgt der linken Wand mithilfe des vorderen linken Sensors.

Dieser Zustand wird genutzt, wenn sich der Roboter nach einer Außenecke zu stark einer linken Wand nähert, bevor der seitliche Sensor die Wand erkennt, aber der vordere Sensor dies bereits tut. Dies verhindert Kollisionen mit der Wand, im Fall, wenn der Roboter sich

schräg der Wand nähert und dies nicht herkömmlich korrigieren kann, weil der seitliche Sensor die Wand noch nicht erfasst hat.

Der Übergang in diesen Zustand erfolgt erst, nachdem der Roboter der Wand bereits zu nahegekommen ist. Dies ist wichtig, da es sonst dazu führen kann, dass der Roboter auf die Wand zusteuert und der Sensor dadurch quasi über die Kante blickt, wodurch er noch stärker in Richtung Wand lenken würde und letztendlich gegen die Wand fährt.

Aktionen:

- Regelt den Abstand zur Wand mithilfe eines PID-Reglers.
- Passt die Geschwindigkeiten der Räder an, um den Abstand zur Wand zu halten.
- Setzt `follow_sensor` auf `self.left`.

Übergänge:

- Zustand 1:
Der Roboter erkennt die linke Wand mit dem linken Sensor und beginnt, dieser zu folgen.
(Steuernachricht: "arrived at left wall, start controlling to it")
- Zustand 0:
Der Roboter erkennt eine unerwartete vordere Wand und startet den Initialisierungsprozess erneut.
(Steuernachricht: "error unexpected front wall, restart init")
- Zustand 6: Der Roboter erkennt eine Kante (keine Wand mehr vorne links) und beginnt, geradeaus zu fahren. (Steuernachricht: "detect edge, start driving forward")

Der Übergang zum Geradeausfahren tritt auf, wenn der Roboter eine Kante erreicht, bevor er die Wand mit dem seitlichen Sensor erkennt. Dies geschieht vor allem, wenn das Ende einer kurzen Wand nach einer Außenecke erreicht wird.

Zustand 22: Rechts diagonal folgen

Beschreibung:

Der Roboter folgt der rechten Wand mithilfe des vorderen rechten Sensors.

Dieser Zustand wird genutzt, wenn sich der Roboter nach einer Außenecke zu stark einer rechten Wand nähert, bevor der seitliche Sensor die Wand erkennt, aber der vordere Sensor dies bereits tut. Dies verhindert Kollisionen mit der Wand, im Fall, wenn der Roboter sich schräg der Wand nähert und dies nicht herkömmlich korrigieren kann, weil der seitliche Sensor die Wand noch nicht erfasst hat.

Der Übergang in diesen Zustand erfolgt erst, nachdem der Roboter der Wand bereits zu nahegekommen ist. Dies ist wichtig, da es sonst dazu führen kann, dass der Roboter auf die Wand zusteuert und der Sensor dadurch quasi über die Kante blickt, wodurch er noch stärker in Richtung Wand lenken würde und letztendlich gegen die Wand fährt.

Aktionen:

- Regelt den Abstand zur Wand mithilfe eines PID-Reglers.
- Passt die Geschwindigkeiten der Räder an, um den Abstand zur Wand zu halten.
- Setzt `follow_sensor` auf `self.right`.

Übergänge:

- Zustand 2:
Der Roboter erkennt die rechte Wand mit dem rechten Sensor und beginnt, dieser zu folgen. (Steuernachricht: "arrived at right wall, start controlling to it")
- Zustand 0:
Der Roboter erkennt eine unerwartete vordere Wand und startet den Initialisierungsprozess erneut.
(Steuernachricht: "error unexpected front wall, restart init")
- Zustand 6:
Der Roboter erkennt eine Kante (keine Wand mehr vorne rechts) und beginnt, geradeaus zu fahren.
(Steuernachricht: "detect edge, start driving forward")

Der Übergang zum Geradeausfahren tritt auf, wenn der Roboter eine Kante erreicht, bevor er die Wand mit dem seitlichen Sensor erkennt. Dies geschieht vor allem, wenn das Ende einer kurzen Wand nach einer Außenecke erreicht wird.

Dateien

Controller.py

Das Controller.py-File enthält die Implementierung verschiedener Steuerungsalgorithmen und Reglerklassen, die zur autonomen Navigation und Regelung eines Roboters verwendet werden. Dieses File umfasst die folgenden Hauptklassen:

PIDController: Implementiert einen PID-Regler (Proportional-Integral-Differential-Regler) mit umfassenden Einstellmöglichkeiten zur Begrenzung der einzelnen Regleranteile. Diese Klasse ermöglicht eine gezielte Anpassung der Regelparameter an spezifische Anwendungen.

ESCController: Implementiert einen Extremum Seeking Controller (ESC), eine adaptive Regelungsmethode zur Optimierung von Systemen ohne explizites Modell. Der ESC arbeitet durch kontinuierliche Störung (Dither-Signal) und passt den Steuerparameter basierend auf der Systemreaktion an.

AutonomousController: Steuert die autonome Bewegung des Roboters anhand verschiedener Algorithmen. Diese Klasse umfasst Implementierungen des Right-Hand-Algorithmus, des Pledge-Algorithmus und des V2-Algorithmus, die jeweils unterschiedliche Ansätze zur Navigation und Hindernisvermeidung bieten.

Klasse PIDController

Die Klasse PIDController implementiert einen PID-Regler mit umfassenden Einstellmöglichkeiten zur Begrenzung der einzelnen Regleranteile. Dies ermöglicht eine gezielte Anpassung der Regelparameter an spezifische Anwendungen.

Ein PID-Regler (Proportional-Integral-Differential-Regler) ist ein Regelalgorithmus, der häufig eingesetzt wird, um Systeme auf einen gewünschten Sollwert zu bringen und dort zu halten. Er besteht aus drei Komponenten:

- Proportionalanteil (P): Reagiert direkt auf die Differenz zwischen Soll- und Istwert.
- Integralanteil (I): Berücksichtigt die Summe vergangener Fehlerwerte und sorgt für eine dauerhafte Korrektur.
- Differentialanteil (D): Reagiert auf die Änderungsgeschwindigkeit des Fehlers und trägt zur Dämpfung bei.

Konstruktor

```
__init__(self, kp, ki, kd, p_max = np.inf, p_min = -np.inf, p_minmax = np.inf, pid_max = np.inf, pid_min = -np.inf, pid_minmax = np.inf, i_max = np.inf, i_min = -np.inf, i_minmax = np.inf, d_max = np.inf, d_min = -np.inf, d_minmax = np.inf)
```

Parameter:

- Kp (Float): Proportionalfaktor, bestimmt die Reaktion auf den aktuellen Fehler.

- `Ki` (Float): Integralanteil, beeinflusst die langfristige Korrektur durch Fehleraufsummierung.
- `Kd` (Float): Differentialanteil, berücksichtigt die Änderungsrate des Fehlers zur Dämpfung.

Zusätzlich können Begrenzungen für die einzelnen Regleranteile gesetzt werden, um unerwünschte Überschwinger oder Instabilitäten zu vermeiden:

Begrenzungen für den Proportionalanteil (P):

- `p_max` (Float): Maximale Begrenzung für den Proportionalanteil.
- `p_min` (Float): Minimale Begrenzung für den Proportionalanteil.
- `p_minmax` (Float): Symmetrische Begrenzung für den Proportionalanteil.

Begrenzungen für den Integralanteil (I):

- `i_max` (Float): Maximale Begrenzung für den Integralanteil.
- `i_min` (Float): Minimale Begrenzung für den Integralanteil.
- `i_minmax` (Float): Symmetrische Begrenzung für den Integralanteil.

Begrenzungen für den Differentialanteil (D):

- `d_max` (Float): Maximale Begrenzung für den Differentialanteil.
- `d_min` (Float): Minimale Begrenzung für den Differentialanteil.
- `d_minmax` (Float): Symmetrische Begrenzung für den Differentialanteil.

Begrenzungen für das Gesamtsignal (PID):

- `pid_max` (Float): Maximale Begrenzung für das gesamte PID-Signal.
- `pid_min` (Float): Minimale Begrenzung für das gesamte PID-Signal.
- `pid_minmax` (Float): Symmetrische Begrenzung für das gesamte PID-Signal.

Hauptmethoden

`update(self, setpoint, measurement, time_step)`

berechnet den neuen Reglerausgang des PID-Reglers

Parameter:

- `Setpoint` (Float): Der gewünschten Sollwert.
- `Measurement` (Float): Der aktuellen Istwert.
- `time_step` (Float): Der vergangenen Zeit seit dem letzten Aufruf.

Rückgabe:

- `PID` (Float): Ausgang des PID-Reglers

Weitere Methoden

`set_previous_error(self, error)`

Setzt die vorherige Regelabweichung, welche für das Berechnen des Differenzials verwendet wird.

Dies kann zur Initialisierung oder Rücksetzung des Reglers genutzt werden.

Parameter:

- `error (Float)`: Die vorherige Regelabweichung

`set_integral(self, integral)`

Ermöglicht das Setzen eines Wertes für das Integralglied, falls eine Anpassung notwendig ist.

Parameter:

- `integral (Float)`: Wert des Integrals

Klasse ESCController

Die Klasse ESCController implementiert einen ESC-Regler, der ursprünglich zur automatischen orthogonalen Ausrichtung zur Wand in der Initialisierungsphase der autonomen Regelung vorgesehen war. Dies konnte jedoch nicht erfolgreich umgesetzt werden, weshalb der ESCController derzeit nicht verwendet wird.

Ein Extremum Seeking Controller (ESC) ist eine adaptive Regelungsmethode, die zur Optimierung von Systemen ohne explizites Modell genutzt wird. Er arbeitet durch kontinuierliche Störung (Dither-Signal) und beobachtet die Reaktion des Systems auf diese Störungen. Basierend auf dieser Rückmeldung passt er den Steuerparameter an, um ein gewünschtes Optimum (z. B. minimale oder maximale Kostenfunktion) zu erreichen.

Konstruktor

`__init__(self, dither_freq, dither_amp, learning_rate)`

Parameter:

- `dither_freq (Float)`: Frequenz des Dithersignals, das zur Anregung des Systems genutzt wird.
- `dither_amp (Float)`: Amplitude des Dithersignals.

- `learning_rate` (Float): Lernrate zur Anpassung des Steuerparameters basierend auf den erfassten Daten.

Hauptmethoden

`update(self, cost, time_step)`

Berechnet den neuen Regelausgang des ESC-Reglers

Parameter:

- `cost` (Float): Die Kosten, welche man minimieren möchte. In diesem Fall ist es der Messwert des vorderen Sensors.
- `time_step` (Float): Die vergangene Zeit seit dem letzten Aufruf.

Rückgabe:

- `control_signal` (Float): Ausgang des ESC-Reglers

Weitere Methoden

`set_previous_hpf_input(cost)`

Setzt den vorherigen Eingabewert für den Hochpassfilter, welcher zur Demodulation der Kosten bzw. Eingangssignals verwendet wird. Dies kann sinnvoll bei initialer Verwendung des Reglers sein.

Parameter:

- `cost` (Float): Der zu setzende Wert.

`set_integrator(self, integrator)`

Ermöglicht das Setzen eines Wertes für den Integrator, welcher zur Integration des demodulierten Signals verwendet wird.

Parameter:

- `integrator` (Float): Der zu setzende Wert.

Klasse AutonomousController

Die Klasse AutonomousController steuert die autonome Bewegung eines Roboters unter Verwendung verschiedener Algorithmen. Sie umfasst drei Implementierungen:

- Right-Hand-Algorithmus: Der zuerst implementierte Algorithmus, bei dem der Roboter immer einer Wand folgt. Diese Implementierung ist weniger optimiert und wurde nicht weiter gepflegt.
- Pledge-Algorithmus: Der eigentliche Hauptalgorithmus zur autonomen Navigation. Er ermöglicht es dem Roboter, Hindernisse zu umgehen und dennoch in die ursprüngliche Richtung zurückzukehren, indem er die Drehungen des Roboters zählt.
- V2-Algorithmus: Eine frühere Version aus der Simulationsumgebung, die als unpraktikabel erachtet wurde und daher nicht implementiert ist.

Konstruktor

- `__init__(self, angle_pid, wall_distance_pid, diag_wall_distance_pid, point_distance_pid, esc, base_speed, base_rotation_speed, desired_distance, sensor_activation_threshold, diagonal_activation_threshold, near_activation_threshold, wheel_distance, robot_radius, sensor_angles, robot, control_distance = 5, angle_toleranz = 3, distance_toleranz = 1.5, esc_angle_comparison_interval = 1, esc_angle_toleranz = 0.8, feature_toleranz = 3, direkt_change_toleranz = 5)`

Pflichtparameter:

- `angle_pid(PIDController)`: PID-Regler für den Winkel.
- `wall_distance_pid(PIDController)`: PID-Regler für den seitlichen Abstand zur Wand beim Fahren mittels eines seitlichen Sensors.
- `diag_wall_distance_pid(PIDController)`: PID-Regler für den seitlichen Abstand zur Wand mittels eines diagonalen Sensors.
- `point_distance_pid(PIDController)`: PID-Regler für den vorderen Abstand zu einem Punkt oder Wand.
- `esc`: Extremum Seeking Controller (ESC).
- `base_speed (Float)`: Basisgeschwindigkeit des Roboters.
- `base_rotation_speed (Float)`: Basisdrehgeschwindigkeit des Roboters.
- `desired_distance (Float)`: Gewünschter Abstand zur Wand.
- `sensor_activation_threshold (Float)`: Schwellenwert zur Aktivierung der Sensoren.
- `diagonal_activation_threshold (Float)`: Schwellenwert zur Aktivierung der diagonalen Sensoren.

- `near_activation_threshold` (Float): Schwellenwert zur Erkennung der Wand als nah.
- `wheel_distance` (Float): Abstand zwischen den Rädern des Roboters.
- `robot_radius` (Float): Radius des Roboters.
- `sensor_angles` (Float): Winkel der Sensoren.
- `robot` (DifferentialDriveRobot): Referenz auf das Roboterobjekt.

Optionale Parameter:

- `control_distance`: Abstand zum Sollabstand ab dem nach vorne zu einer Wand oder Punkt geregelt werden soll. (Standardwert: 5)
- `angle_toleranz`: verwendet beim Drehen im Stand, um zu bestimmen, wann der Sollwert erreicht wurde. (Standardwert: 3)
- `distance_toleranz`: verwendet bei der Regelung zu einem Abstand nach vorne, um zu bestimmen, wann der Sollwert erreicht wurde. (Standardwert: 1.5)

Parameter die nicht mehr verwendet werden:

- `esc_angle_comparison_interval`: Intervall für den Vergleich des ESC-Winkels.
- `esc_angle_toleranz`: Toleranz für den ESC-Winkel.
- `feature_toleranz`: Toleranz für Merkmale.
- `direkt_change_toleranz`: Toleranz für direkte Änderungen.

Der Konstruktor initialisiert weitere wichtige Parameter:

- `self.undercut`: modelliert den Winkel bei 90° links oder rechts drehen.
- `self.point_overshoot`: Modelliert die Position des Zielpunktes.
- `self.kw_standing`: Koeffizient der Linearen rückführung der Drehgeschwindigkeit beim im Stand drehen.
- `self.kw_driving`: Koeffizient der Linearen rückführung der Drehgeschwindigkeit beim geradeaus fahren.
- `self.front_desired_distance_faktor`: Faktor für den gewünschten Abstand nach vorne.
- `self.corner_init_front_desired_distance_faktor`: Faktor für den initialen gewünschten Abstand in Ecken.

Hauptmethoden

In diesen Methoden sind die jeweiligen Algorithmen zur autonomen Steuerung implementiert und haben die folgenden Parameter und Rückgabewerte:

Parameter:

- `sensor_readings` (Liste von Float): Liste der aktuellen Sensordaten
- `x, y, theta` (Float): Neue Position und Winkel

- `omega` (Float): Drehgeschwindigkeit
- `current_time` (Float): Aktueller Zeitstempel.
- `time_step` (Float): Die vergangene Zeit seit dem letzten Aufruf.

Rückgabe:

- `left_wheel_velocity` (Float): Sollwert der linken Radgeschwindigkeit.
- `right_wheel_velocity` (Float): Sollwert der rechten Radgeschwindigkeit.

`autonomous_control_right_hand(self, sensor_readings, x, y, theta, omega, current_time, time_step):`

Implementiert die autonome Steuerung des Roboters unter Verwendung des Right-Hand-Algorithmus. Der Right-Hand-Algorithmus gewährleistet, dass der Roboter stets der rechten Wand folgt. Dieser einfache Algorithmus nutzt die Sensordaten, um den Abstand zur Wand konstant zu halten und den Roboter entlang der Wand zu navigieren.

`autonomous_control_pledge(self, sensor_readings, x, y, theta, omega, current_time, time_step):`

Implementiert die autonome Steuerung des Roboters unter Verwendung des Pledge-Algorithmus. Der Pledge-Algorithmus ermöglicht es dem Roboter, Hindernisse zu umfahren und dennoch in die ursprüngliche Richtung zurückzukehren. Dies wird durch das Zählen der Drehungen des Roboters erreicht. Der Algorithmus stellt sicher, dass der Roboter nicht in Schleifen gefangen bleibt und garantiert das Finden des Ausgangs.

`autonomous_control_V2(self, sensor_readings, x, y, theta, current_time, time_step):`

Der V2-Algorithmus ist eine selbst erdachte erweiterte Version des Right-Hand-Algorithmus. Zusätzlich zur Navigation entlang der rechten Wand versucht dieser Algorithmus zu vermeiden, dass der Roboter an einer freistehenden Wand im Labyrinth unendlich lange entlangfährt. Wenn der Roboter an einer Stelle ist, an der er bereits gewesen ist, wechselt er die Wand, um eine Endlosschleife zu vermeiden.

Getter- und Setter-Methoden

- `get_base_speed(self)`: (Float) Gibt die Basisgeschwindigkeit zurück.
- `get_state(self)`: (Int) Gibt den aktuellen Zustand zurück.
- `get_follow_sensor(self)`: (Int) Gibt den aktuellen Folgesensor zurück.
- `get_on_closed_loop(self)`: (Bool) Gibt die `on_closed_Loop`-Variable zurück. (Ist nur für V2 relevant)
- `get_set_angle_set_point(self)`: (Float) Gibt den Sollwinkel zurück.
- `get_pledge_count(self)`: (Int) Gibt den Pledgezähler zurück.

- `set_base_speed(self, base_speed)`: (Float) Setzt die Basisgeschwindigkeit.

Weitere Methoden

`relative_angle(self, robot_x, robot_y, robot_orientation, target_x, target_y)`:

Berechnet den absoluten Winkel zum Zielpunkt, unter Berücksichtigung der Rotationen des Roboters, unabhängig von der Drehrichtung (positiv oder negativ).

Parameter:

- `robot_x` (Float): Aktuelle x-Koordinate des Roboters.
- `robot_y` (Float): Aktuelle y-Koordinate des Roboters.
- `robot_orientation` (Float): Aktuelle Winkel (rad).
- `target_x` (Float): x-Koordinate des Zielpunkts.
- `target_y` (Float): y-Koordinate des Zielpunkts.

Rückgabe:

- `absolute_angle` (Float): Absoluter Winkel zum Zielpunkt (rad).

`flanke_detektion(self, sensor_value, prev_value, threshold = 151)`:

Erkennt, ob der Sensor etwas detektiert, und überprüft Flanken im Status des Sensors basierend auf einem Schwellenwert.

Parameter:

- `sensor_value` (Int): Aktueller Wert des Sensors.
- `prev_value` (Int): Vorheriger Aktivierungszustand des Sensors.
- `threshold` (Int)(Optional): Schwellenwert zur Aktiverkennung.

Rückgabewert:

- `sensor_aktive` (Bool): True, wenn der Sensorwert unterhalb des Schwellwert ist.
- `sensor_flanke` (Bool): True, wenn eine Flanke detektiert wurde, sonst False.

`get_distance_to_point(self, robot_x, robot_y, robot_orientation, target_x, target_y)`:

Berechnet den Abstand zwischen dem Roboter und einem gegebenen Punkt. Der Abstand wird negativ zurückgegeben, wenn der Punkt hinter dem Roboter liegt.

Parameter:

- `robot_x` (Float): Aktuelle x-Koordinate des Roboters.
- `robot_y` (Float): Aktuelle y-Koordinate des Roboters.
- `robot_orientation` (Float): Aktuelle Orientierung des Roboters in Grad.
- `target_x` (Float): x-Koordinate des Zielpunkts.

- `target_y` (Float): y-Koordinate des Zielpunkts.

Rückgabewert:

- `distance` (Float): Abstand zum Zielpunkt. Negativ, wenn der Punkt hinter dem Roboter liegt.

`get_forward_point(self, x, y, theta, distance):`

Berechnet die Koordinaten eines Punktes, der sich in einer bestimmten Distanz in Fahrtrichtung des Roboters befindet.

Parameter:

- `x` (Float): Aktuelle x-Koordinate des Roboters.
- `y` (Float): Aktuelle y-Koordinate des Roboters.
- `theta` (Float): Aktuelle Orientierung des Roboters in Grad.
- `distance` (Float): Distanz in Fahrtrichtung.

Rückgabewert:

- (Tuple[Float, Float]): Koordinaten (x, y) des berechneten Punktes.

`has_angle_changed_less_than_threshold(self, previous_theta, current_theta, esc_angle_toleranz):`

Überprüft, ob sich der Winkel nur geringfügig geändert hat, indem die Änderung des Winkels zwischen zwei Zeitpunkten berechnet und mit einer Toleranz verglichen wird. (Wurde zu Konvergenzüberprüfung des ESC verwendet)

Parameter:

- `previous_theta` (Float): Vorheriger Winkel.
- `current_theta` (Float): Aktueller Winkel.
- `esc_angle_toleranz` (Float): Toleranzwert für die Winkeländerung.

Rückgabewert:

- (Bool): True, wenn die Winkeländerung innerhalb der Toleranz liegt, sonst False.

`reset(self, angle_setpoint=None):`

Setzt den Zustand des Reglers zurück und setzt dabei relevante Parameter. Optional kann ein neuer Sollwinkel gesetzt werden

Parameter:

- `angle_setpoint` (Float) (Optional): Neuer Sollwinkel.

`check_features(self, x, y, theta):`

Überprüft, ob die aktuellen Koordinaten des Roboters ein neues Merkmal darstellen oder ob der Roboter sich an einer bereits besuchten Stelle befindet. (wurde für V2 verwendet)

Parameter:

- `x (Float)`: Aktuelle x-Koordinate des Roboters.
- `y (Float)`: Aktuelle y-Koordinate des Roboters.
- `theta (Float)`: Aktuelle Winkel des Roboters.

`check_change_features(self):`

Überprüft, ob das vorherige Merkmal ein neues Wechselmerkmal darstellt oder ob es sich um ein bereits besuchtes Wechselmerkmal handelt. (Wurde für V2 verwendet)

Rückgabewert:

- (Bool): True, wenn ein neues Wechselmerkmal erkannt wurde, sonst False.

`get_sensor_index(self, sensor_angles, direction):`

Gibt den Index des Sensors zurück, der am nächsten an einer bestimmten Richtung liegt.

Parameter:

- `sensor_angles (List[Float])`: Liste der Sensorwinkel in Rad.
- `direction (String)`: Zielrichtung 'left', 'front_left', 'front', 'front_right', 'right'

Rückgabewert:

- `closest_index (Int)`: Index des Sensors, der am nächsten an der Zielrichtung liegt.

filter.py

Diese Datei implementiert zwei digitale Filter (Tiefpass- und Hochpassfilter) zur Signalverarbeitung sowie eine funktionsfähige, jedoch nicht verwendete Implementierung eines Unscented Kalman Filters (UKF). Der Tiefpassfilter dient zur Glättung von Signalen, während der Hochpassfilter hochfrequente Anteile hervorhebt. Obwohl der UKF grundsätzlich zur Zustandsschätzung geeignet ist, wurde er nicht genutzt, da die Zustände bereits hinreichend genau ohne ihn geschätzt werden konnten. Zudem war die Parametrisierung der Prozessrausch-Kovarianzmatrix des UKF unklar, da mir die Varianzen des Systems nicht bekannt sind.

Klasse LowPassFilter

Diese Klasse implementiert einen digitalen Tiefpassfilter zur Glättung von Signalen.

Konstruktor

```
__init__(self, cutoff_freq)
```

Parameter:

- `cutoff_freq` (Float): Grenzfrequenz des Filters in Hertz.

Hauptmethoden

```
filter(self, input_signal, time_step)
```

Wendet den Tiefpassfilter auf das Eingangssignal an.

Parameter:

- `input_signal` (Float): Das zu filternde Signal.
- `time_step` (Float): Der Zeitschritt.

Rückgabe:

- Float: Das gefilterte Signal.

```
set_previous_output(self, output)
```

Setzt den vorherigen Ausgangswert für die nächste Filteriteration.

Parameter:

- `output` (Float): Neuer vorheriger Ausgangswert.

Klasse HighPassFilter

Diese Klasse implementiert einen digitalen Hochpassfilter zur Hervorhebung von hochfrequenten Signalanteilen.

Konstruktor

`__init__(self, cutoff_freq)`

Parameter:

- `cutoff_freq` (Float): Grenzfrequenz des Filters in Hertz.

Hauptmethoden

`filter(self, input_signal, sampling_period)`

Wendet den Hochpassfilter auf das Eingangssignal an.

Parameter:

- `input_signal` (Float): Das zu filternde Signal.
- `sampling_period` (Float): Die Abtastperiode des Signals.

Rückgabe:

- `output_signal` (Float): Das gefilterte Signal.

Weitere Methoden

`set_previous_input(self, input)`

Setzt den vorherigen Eingangswert für die nächste Filteriteration.

Parameter:

- `input` (Float): Neuer vorheriger Eingangswert.

Klasse UKFEstimator

Diese Klasse implementiert einen Unscented Kalman Filter (UKF) zur Zustandsschätzung eines Systems. Sie nutzt die filterpy-Bibliothek zur Berechnung und Verwaltung des Filters.

Konstruktor

`__init__(self, dt, initial_state, process_noise, measurement_noise, motion_model, measurement_model)`

Parameter:

- `dt` (Float): Zeitschritt zwischen den Messungen.
- `initial_state` (Array): Anfangszustand des Systems.
- `process_noise` (np.ndarray): Kovarianzmatrix des Prozessrauschens.
- `measurement_noise` (np.ndarray): Kovarianzmatrix des Messrauschens.
- `motion_model` (Funktion): Modell, das die Systemdynamik beschreibt.
- `measurement_model` (Funktion): Modell, das die Messwerte simuliert.

Der Konstruktor initialisiert den UKF mit den gegebenen Modellen, Rauschparametern und einem Sigma-Punkt-Generator.

Hauptmethoden

`predict(self, dt)`

Führt einen Zeitschritt der UKF-Prädiktion durch.

Parameter:

- `dt` (Float): Zeitschritt für die Vorhersage.

`update(self, measurements)`

Aktualisiert die Systemzustände basierend auf neuen Messwerten.

Parameter:

- `measurements` (Array): Die aktuellen Messwerte.

`get_state(self)`

Gibt die geschätzten Zustände zurück.

Rückgabe:

- (Tuple[Float, Float, ...]): Die geschätzten Zustände, Anzahl und Reihenfolge der Zustände sind vom Bewegungsmodell abhängig.

`set_state(self, state)`

Setzt die aktuellen geschätzten Zustände auf einen neuen Wert.

Parameter:

- (Tuple[Float, Float, ...]): Die zu setzenden Zustände, Anzahl und Reihenfolge der Zustände sind vom Bewegungsmodell abhängig.

lo_management.py

Die Datei umfasst Mechanismen für die Konsolenausgabe (inklusive Logging der Steuerinformationen) sowie für die Visualisierung der Radgeschwindigkeiten. Dabei gibt es zwei Arten von Plots: den Echtzeitplot, der die Entwicklung der Radgeschwindigkeiten über die Zeit visualisiert, und den Batch-Plot, der nach Beendigung der Fahrt erstellt wird. Diese dienen vor allem dem Debugging. Der Echtzeitplotter wurde jedoch kaum genutzt, da er zu viel Latenz in das System brachte.

Darüber hinaus wird eine Funktion zur kontinuierlichen Erfassung von Benutzereingaben (Tastatureingaben) bereitgestellt, die es ermöglicht, den Roboter manuell zu steuern

Klasse OutputManager

Die Klasse OutputManager dient als zentrales Interface für alle Ein-/Ausgabeoperationen. Sie koordiniert den Start und Stopp der Konsolenausgabe (über ein Objekt der Klasse ConsoleOutput) sowie der Echtzeit- und Batch-Plots. Außerdem ermöglicht sie das Aktualisieren und Anzeigen der Log-Daten.

Konstruktor

`__init__(self)`

Initialisiert den OutputManager.

Hauptmethoden

`start_console_output(auto_controller=None, extra_thread=True)`

Diese Methode des erstellt zunächst ein neues Objekt der Klasse ConsoleOutput.

Parameter:

- `auto_controller` (AutonomousController): Wird benötigt falls die speziellen Konsolenausgaben für den autonomen Modus verwendet werden sollen.
- `extra_thread`: Legt fest, ob die Konsolenausgabe in einem eigenen Thread ausgeführt werden soll.

Ist `extra_thread` auf `True` gesetzt, so wird die Methode `start()` der `ConsoleOutput`-Klasse intern aufgerufen, wodurch ein separater Thread gestartet wird, der fortlaufend die Methode `display_console_output()` aufruft und damit den aktuellen Konsolenoutput automatisch aktualisiert. Ist `extra_thread` auf `False` gesetzt, wird lediglich das `ConsoleOutput`-Objekt erzeugt, ohne dass ein eigener Thread gestartet wird. In diesem Fall muss der Benutzer die Methode `display_console_output()` manuell aufrufen, um den Output in der Konsole anzuzeigen.

`update_console_output(self, robot, left_wheel_velocity, right_wheel_velocity, base_speed, angle_setpoint, angle_control, pid_r, pid_l, sensor_readings, auto_active=False)`

Diese Methode aktualisiert den Inhalt der Konsolenausgabe. Sie sammelt aktuelle Daten (wie z. B. Ist- und Sollwerte der Radgeschwindigkeiten, Winkel, PID-Reglerwerte und Sensordaten) und übergibt diese an das ConsoleOutput-Objekt.

Parameter:

- `robot` (DifferentialDriveRobot-Objekt): Das Roboterobjekt, aus dem Positions- und Zustandsdaten abgerufen werden.
- `left_wheel_velocity` (float): Sollwert der linken Radgeschwindigkeit.
- `right_wheel_velocity` (float): Sollwert der rechten Radgeschwindigkeit.
- `base_speed` (float): Basisgeschwindigkeit des Roboters.
- `angle_setpoint` (float): Gewünschter Sollwinkel (in Radiant) bzw. Richtungsänderung.
- `angle_control` (float): Steuerungsanteil zur Winkelkorrektur, berechnet durch den PID-Regler.
- `pid_r` (PIDController-Objekt): PID-Regler für die rechte Seite.
- `pid_l` (PIDController-Objekt): PID-Regler für die linke Seite.
- `sensor_readings` (Liste von float): Liste der aktuellen Sensordaten.
- `auto_active` (bool, optional): Gibt an, ob der autonome Modus ausgaben verwendet werden sollen.

`stop_console_output(self)`

Beendet die laufende Konsolenausgabe, indem der zugehörige Thread gestoppt wird (falls vorhanden).

`update_plot(self, current_time, left_wheel_velocity, right_wheel_velocity, left_wheel_velocity_target, right_wheel_velocity_target)`

Diese Methode übergibt aktuelle Mess- und Sollwerte an das RealTimePlotter-Objekt, sodass der Plot laufend aktualisiert wird.

Parameter:

- `current_time` (float): Aktueller Zeitstempel (in Sekunden).
- `left_wheel_velocity` (float): Aktueller Ist-Wert der linken Radgeschwindigkeit.
- `right_wheel_velocity` (float): Aktueller Ist-Wert der rechten Radgeschwindigkeit.
- `left_wheel_velocity_target` (float): Sollwert der linken Radgeschwindigkeit.
- `right_wheel_velocity_target` (float): Sollwert der rechten Radgeschwindigkeit.

`start_rt_plot(self, rt_window_size=10)`

Initialisiert ein RealTimePlotter-Objekt, das die Echtzeitvisualisierung der Radgeschwindigkeiten übernimmt.

Parameter:

- `rt_window_size` (int, optional): Zeitfenster für die Echtzeitanzeige.

`activate_final_batch_plot(self)`

Setzt das Flag `final_batch_plot_active` auf True, sodass beim Programmende ein Batch-Plot durch `show_final_plots()` angezeigt wird.

`activate_final_rt_plot(self)`

Setzt das Flag `final_rt_plot_active` auf True, sodass beim Programmende der finale Echtzeit-Plot durch `show_final_plots()` angezeigt wird.

`show_final_plots(self, robot=None)`

Zeigt abhängig von den gesetzten Flags entweder den finalen Echtzeit-Plot oder einen Batch-Plot an. Falls `final_rt_plot_active` nicht True ist, beendet die Funktion den RT-Plot.

Parameter:

- `robot` (DifferentialDriveRobot-Objekt, optional): Das Roboterobjekt, aus dem die vollständigen Zeitreihen (Mess- und Sollwerte) abgerufen werden. Nur für Batch-Plot notwendig.

`show_full_log(self)`

Diese Methode liest den gesamten Inhalt der Log-Datei aus und zeigt ihn in der Konsole an.

Weitere Methoden

`show_batch_plot(self, robot)`

Erzeugt und zeigt einen Batch-Plot in einem separaten Fenster, der die gesamte Aufzeichnung der Radgeschwindigkeiten (Ist- und Sollwerte) über die gesamte Fahrzeit darstellt.

`show_final_rt_plot(self)`

Zeigt den finalen Echtzeit-Plot an, wenn dieser final angezeigt werden soll.

Klasse ConsoleOutput

Diese Klasse wird von der Outputmanager Klasse koordiniert und ist dafür zuständig, die gesamte Konsolenausgabe der Robotersteuerung in regelmäßigen Abständen zu

aktualisieren und darzustellen. Dabei wird, sobald sich die aktuell ausgegebene Steuernachricht (`control_message`) ändert, der komplette formatierte Konsolenoutput (`info_lines`) in eine Log-Datei geschrieben. Dies ermöglicht es, den Systemverlauf des aktuellen Programmablaufs für spätere Analysen einzusehen.

Konstruktor

`__init__(self, auto_controller)`

Initialisiert das `ConsoleOutput`-Objekt und setzt interne Variablen, die zur Verwaltung der Konsolenausgabe verwendet werden.

Parameter:

- `auto_controller` (`AutonomousController`-Objekt): Objekt, das erweiterte Informationen (wie Zustandsinformationen, Folgesensor, Pledge-Zähler und zugehörige Steuernachrichten) im autonomen Modus liefert. Diese Informationen werden in den Konsolenoutput integriert, wenn der autonome Modus aktiv ist.

Methoden

`start(self)`

Startet den Konsolenoutput. Diese Methode startet einen eigenen Thread, der die Methode `run_console_output()` ausführt. Dadurch wird der Konsolenoutput asynchron aktualisiert.

`stop(self)`

Beendet den Konsolenoutput, indem der in `start()` gestartete Thread gestoppt und beendet wird.

`run_console_output(self)`

Diese Methode wird in einem separaten Thread ausgeführt und sorgt dafür, dass der Konsolenoutput kontinuierlich in regelmäßigen Abständen (etwa alle 0,05 s) durch Aufruf der Methode `display_console_output()` aktualisiert wird.

`update(self, robot, left_wheel_velocity, right_wheel_velocity, base_speed, angle_setpoint, angle_control, pid_r, pid_l, sensor_readings, auto_active=False)`

Aktualisiert den internen Konsolenoutput (`info_lines`) mit den neuesten Daten.

Im autonomen Modus (`auto_active = True`) werden spezielle Werte (wie z. B. der aktuelle Zustand, Folgesensor, Pledge-Zähler und die zugehörige Steuernachricht) aus dem übergebenen `AutonomousController`-Objekt übernommen und formatiert.

Im manuellen Modus werden stattdessen Standarddaten wie Ist- und Sollwerte der Radgeschwindigkeiten, PID-Reglerausgänge sowie Sensordaten ausgegeben.

Nach jeder Aktualisierung prüft die Methode, ob sich die Steuernachricht (`control_message`) geändert hat. Falls ja, wird der gesamte, formatierte Konsolenoutput in die Log-Datei geschrieben.

Parameter:

- `robot` (`DifferentialDriveRobot`): Das Roboterobjekt, aus dem aktuelle Positions- und Zustandsdaten abgerufen werden.
- `left_wheel_velocity` (`float`): Sollwert der linken Radgeschwindigkeit.
- `right_wheel_velocity` (`float`): Sollwert der rechten Radgeschwindigkeit.
- `base_speed` (`float`): Basisgeschwindigkeit des Roboters.
- `angle_setpoint` (`float`): Gewünschter Sollwinkel (in Radiant).
- `angle_control` (`float`): Steuerungsanteil zur Winkelkorrektur, wie er vom PID-Regler berechnet wird.
- `pid_r` (`PIDController`): PID-Regler-Objekt für die rechte Seite.
- `pid_l` (`PIDController`): PID-Regler-Objekt für die linke Seite.
- `sensor_readings` (`Liste[float]`): Liste der aktuellen Sensordaten.
- `auto_active` (`bool`, optional): Gibt an, ob der autonome Modus aktiv ist.

`log_info_lines(self)`

Schreibt den aktuell formatierten Konsolenoutput (`info_lines`) in die Log-Datei.

`display_console_output(self)`

Gibt den aktuell formatierten Konsolenoutput in der Konsole aus. Vor jeder Ausgabe wird der bisherige Konsoleninhalt mittels ANSI Escape-Sequenz gelöscht, sodass stets nur die aktuellen Informationen angezeigt werden.

`show_full_log(self)`

Liest den gesamten Inhalt der Log-Datei aus und zeigt diesen in der Konsole an.

Klasse RealTimePlotter

Diese Klasse wird von der Outputmanager Klasse koordiniert und ist verantwortlich für die Echtzeitvisualisierung der Radgeschwindigkeiten des Roboters. Mithilfe eines gleitenden Zeitfensters (Time Window) werden die Ist- und Sollwerte der linken und rechten Radgeschwindigkeiten in einem interaktiven Plot dargestellt.

Konstruktor

`__init__(self, time_window=10)`

Initialisiert das RealTimePlotter-Objekt. Es werden interne Datenstrukturen zur Speicherung von Zeitstempeln sowie Ist- und Sollwerten der Radgeschwindigkeiten angelegt. Der interaktive Plot wird mithilfe von Matplotlib erstellt und konfiguriert.

Parameter:

- `time_window` (Float): Das Zeitfenster in Sekunden, innerhalb dessen die Datenpunkte im Echtzeitplot angezeigt werden sollen.

Methoden

`update_plot(self, current_time, left_velocity, right_velocity, left_target, right_target)`

Diese Methode aktualisiert den Plot, indem sie die aktuellen Zeit- und Geschwindigkeitswerte in die internen Deques speichert. Es werden nur die Datenpunkte innerhalb des vorgegebenen Zeitfensters (`time_window`) berücksichtigt. Anschließend werden die Linien im Plot mit den neuen Daten aktualisiert und die Achsen (X und Y) entsprechend skaliert.

Parameter:

- `current_time` (float): Aktueller Zeitstempel (in Sekunden).
- `left_velocity` (float): Aktueller Ist-Wert der linken Radgeschwindigkeit.
- `right_velocity` (float): Aktueller Ist-Wert der rechten Radgeschwindigkeit.
- `left_target` (float): Sollwert der linken Radgeschwindigkeit.
- `right_target` (float): Sollwert der rechten Radgeschwindigkeit.

`show(self)`

Zeigt den interaktiven Plot an, ohne den Hauptsteuerungszyklus zu blockieren.

`show_final_plot(self)`

Schaltet den interaktiven Modus von Matplotlib ab und zeigt den finalen Plot an.

`stop(self)`

Schließt das Plot-Fenster, sodass der Plot nicht länger im Vordergrund angezeigt wird. Diese Methode wird auch automatisch beim Löschen des Objekts aufgerufen.

`__del__(self)`

Destructor der Klasse, der sicherstellt, dass das Plot-Fenster beim Löschen des Objekts geschlossen wird.

Funktion: `handle_user_input (angle_setpoint, base_speed, autonomous_mode, close=False, debounce_delay=0.2, reset=None)`

Die Funktion `handle_user_input` verarbeitet Tastatureingaben. Dabei werden diskrete Aktionen, wie das Umschalten des autonomen Modus oder das Setzen eines Abbruchsignals, durch einen Entprellmechanismus nur einmal pro Tastendrückung ausgeführt. Gleichzeitig können kontinuierliche Anpassungen vorgenommen werden, wenn Tasten (z. B. „up“, „down“, „left“, „right“) gedrückt gehalten werden bei der manuellen Steuerung des Roboters.

Parameter:

- `angle_setpoint (float)`:
Diente als aktueller Winkel-Setpoint des Roboters zur Steuerung per Winkelvorgabe und Tangentialer Geschwindigkeit und wurde durch die Tasten „left“ bzw. „right“ erhöht oder verringert. Dieser Wert wird jedoch derzeit intern zurückgesetzt und ist somit wirkungslos.
- `base_speed (float)`:
Dieser Wert diente als Ausgangswert der Basisgeschwindigkeit und wurde durch die Tasten „up“ bzw. „down“ erhöht oder verringert. Dieser Wert wird jedoch derzeit intern Zurückgesetzt und somit wirkungslos.
- `autonomous_mode (bool)`:
Ein Flag, das angibt, ob der autonome Modus aktiviert ist (True) oder nicht (False). Mit der Taste „a“ wird dieser Modus umgeschaltet (Toggle).
- `close (bool, optional)`:
Ein Flag (Standard: False), das auf True gesetzt wird, wenn die Taste „c“ gedrückt wird, um das Beenden des Programms zu signalisieren.

- `debounce_delay` (float, optional):
Die Mindestzeit (in Sekunden), die zwischen diskreten Tastendrücken vergehen muss, um ein Entprellen zu gewährleisten (Standard: 0.2 Sekunden).
- `reset` (Callable, optional):
Hier kann die Reset-Funktion des `AutonomousController`-Objekts übergeben werden, sodass der Zustandsautomat zurückgesetzt wird, wenn man zwischen manuellem und autonomem Modus wechselt. Wenn diese Funktion nicht gesetzt ist, möchte der Roboter im autonomen Modus genau dort weitermachen, wo er aufgehört hat.

Rückgabe:

- `angle_setpoint` (float):
Der aktuelle Radgeschwindigkeit, welche auf die Basisgeschwindigkeit aufaddiert wird, um den Roboter sich drehen zu lassen. Dieser Wert durch Tastatureingaben (links/rechts) fest gesetzt solange die jeweilige taste gehalten wird.
- `base_speed` (float):
Die aktuelle Basisgeschwindigkeit des Roboters. Dieser Wert wird durch die Tasten „up“ bzw. „down“ fest gesetzt solange die jeweilige Taste gehalten wird.
- `close` (bool):
Ein Flag (Standard: False), das auf True gesetzt wird, wenn die Taste „c“ gedrückt wird, um das Beenden des Programms zu signalisieren.
- `autonomous_mode` (bool):
Ein Flag, das angibt, ob der autonome Modus aktiviert ist (True) oder nicht (False). Mit der Taste „a“ wird dieser Modus umgeschaltet (Toggle).

robot.py

Diese Datei beinhaltet lediglich die Klasse `DifferentialDriveRobot`. Es ist jedoch vorgesehen, dass im Falle einer anderen Hardware diese hier als neue Klasse implementiert wird und man dann zum Wechseln nur das Objekt der neuen Hardware erzeugt, statt `DifferentialDriveRobot`. Derzeit unterstützt die autonome Steuerung jedoch nur einen Roboter mit Differentialantrieb. Daher gilt das einfache Austauschen derzeit nur für Hardware mit dieser Antriebstechnik.

Klasse `DifferentialDriveRobot`

Die Klasse bildet das zentrale Interface zur Ansteuerung des Roboters und zur Herstellung der Verbindung zur Hardware. Sie dient primär dazu, das Robotermodell, dessen Parameter und zusätzliche Statusdaten zu verwalten sowie die Kommunikation mit den angeschlossenen Komponenten (Sensoren, Motoren, Encodern, I²C-Geräten usw.) zu ermöglichen. Dies umfasst unter anderem das Einlesen von Sensordaten, die Umrechnung von Spannungen in Distanzen und die Ansteuerung der Motoren über spezialisierte Bibliotheken. Der Roboter wird über einen Raspberry Pi 4 gesteuert. In diesem Kontext werden verschiedene Schnittstellen (wie `pigpio`, `Motoron-I2C`, `ADCDifferentialPi`, `INA219`, `adafruit_icm20x`, `board`) verwendet, um die Hardwarekomponenten zu initialisieren und anzusteuern.

Konstruktor

```
__init__(self, param_file='parameters.txt', mesurment_noise_mean=0,
mesurment_noise_standard_deviation=1, system_noise_mean=0,
system_noise_standard_deviation=1, init_robot_x=0, init_robot_y=0, init_robot_angle=0,
motion_model=None, measurement_model=None, process_noise=None,
measurement_noise=None, dt=0.081)
```

Der Konstruktor initialisiert sämtliche Hardware-Verbindungen, Sensoren, Filter und das Robotermodell. Er lädt auch ein Parameterfile, das zur Umrechnung der ADC-Spannungen in Distanzen genutzt wird. Die Klasse speichert zudem interne Statusvariablen.

Parameter:

- `param_file` (str, optional):
Dateipfad zur Parameterdatei, die die Koeffizienten zur Umrechnung der gemessenen ADC-Spannung in Distanzen enthält. Mit diesem Parameter kann der Nutzer einen anderen Satz von Umrechnungsparametern laden, als der Standardpfad.
- `mesurment_noise_mean` (float):
Mittelwert des zusätzlichen Messrauschens. Dieser Parameter wird ausschließlich in

der Funktion `get_sensor_readings_with_noise` verwendet, um den ausgelesenen Sensordaten (die bereits umgerechnete Abstände darstellen) zusätzlich Rauschen hinzuzufügen.

- `mesurment_noise_standard_deviation` (float):
Standardabweichung des zusätzlichen Messrauschens. Auch dieser Wert dient ausschließlich der Funktion `get_sensor_readings_with_noise` und beeinflusst, wie stark die Sensordaten zusätzlich verrauscht werden.
- `system_noise_mean` (float):
Mittelwert des Systemrauschens, der ausschließlich in der Funktion `update_robot_with_noise` zur Simulation von zusätzlichen Störeinflüssen bei der Zustandsabschätzung verwendet wird.
- `system_noise_standard_deviation` (float):
Standardabweichung des Systemrauschens, der ausschließlich in der Funktion `update_robot_with_noise` zur Simulation von zusätzlichen Störeinflüssen bei der Zustandsabschätzung verwendet wird.
- `init_robot_x` (float): Initiale x-Position des Roboters in Metern.
- `init_robot_y` (float): Initiale y-Position des Roboters in Metern.
- `init_robot_angle` (float): Initiale Orientierung des Roboters in Radiant.
- `motion_model` (Callable, optional):
Beschreibt das Bewegungsmodell des Roboters welches für die UKF-Prediction verwendet wird. Standardmäßig wird das in der Klasse definierte `ukf_motion_model` verwendet jedoch kann hier ein alternatives Modell vorgegeben werden.
- `measurement_model` (Callable, optional):
Beschreibt das Messmodell für den UKF. Standardmäßig wird das in der Klasse definierte `ukf_measurement_model` verwendet jedoch kann hier ein alternatives Modell vorgegeben werden.
- `process_noise` (np.ndarray, optional):
Kovarianzmatrix des Prozessrauschens für den UKF.
- `measurement_noise` (np.ndarray, optional):
Kovarianzmatrix des Messrauschens für den UKF.
- `dt` (float, optional): Zeitintervall in Sekunden für die UKF-Prediction.

Hardware- und Systeminitialisierung

Encoder & GPIO-Konfiguration:

Die GPIO-Pins für die Encoder (für linke und rechte Räder) werden über die pigpio-Bibliothek als Eingänge mit aktivierten Pull-ups konfiguriert.

Linker Encoder:

- Pin A: GPIO 17
- Pin B: GPIO 27

Rechter Encoder:

- Pin A: GPIO 21
- Pin B: GPIO 20

Abhängig vom gewählten `encoder_mode` werden Callback-Funktionen entweder für die Geschwindigkeitsmessung oder die Impulszählung registriert.

Es sind zwei Verfahren zur Bestimmung der Radgeschwindigkeit implementiert, die über die Variable `encoder_mode` gesteuert werden. Aufgrund der Rauschanfälligkeit bei Richtungsangaben wird in dieser Implementierung ausschließlich der Betrag der Geschwindigkeit gemessen, da eine Angabe beider Vorzeichen zu spontanen und rauschbedingten Vorzeichenwechseln führte.

- Encoder-Modus 0 – Zeitintervall-basierte Geschwindigkeitsberechnung

In diesem Modus wird die Geschwindigkeit direkt aus der Zeitdifferenz zwischen zwei aufeinanderfolgenden Impulsen (Rising Edges) berechnet. Bei jedem erfassten Impuls wird der aktuelle Zeitpunkt gespeichert. Die Zeitdifferenz zwischen dem aktuellen und dem vorherigen Impuls wird ermittelt. Anschließend wird die Geschwindigkeit als Quotient aus dem Radumfang (berechnet über die Raddiameter) und dem Produkt aus der Anzahl der Pulse pro Umdrehung (ppr) und dem gemessenen Zeitintervall berechnet. Wird für ein Zeitintervall größer als ein vordefiniertes Timeout, wenn keine Impulse mehr empfangen werden, wird die Geschwindigkeit auf 0 gesetzt.

- Encoder-Modus 1 – Impulszählung und RPS-Berechnung

In diesem Modus werden die an den Encoder-Pins empfangenen Impulse in einem Zähler (`counter_left` bzw. `counter_right`) summiert. Über einen definierten Zeitabschnitt (Zeitstempel bzw. `time_step`) wird die Anzahl der Impulse gezählt und

in Umdrehungen pro Sekunde (RPS) umgerechnet, indem die Anzahl der Pulse durch die Pulsanzahl pro Umdrehung (ppr) geteilt wird. Anschließend wird die Radgeschwindigkeit als Produkt aus dem Radumfang und den berechneten Umdrehungen pro Sekunde ermittelt. Nach der Berechnung werden die Zähler und der zugehörige Zeitstempel zurückgesetzt, um im nächsten Intervall wieder eine frische Messung zu ermöglichen.

Dabei zeigt die bisher getestete Implementierung, dass der Encoder-Modus 0, der auf der direkten Zeitintervallmessung basiert, im Vergleich zum Impulszählverfahren weniger anfällig gegenüber Rauschen ist und daher empfohlen wird. Daher ist dieser auch gesetzt.

ADC (Analog-Digital-Converter):

Ein Objekt der Klasse ADCDifferentialPi wird erstellt, um die analogen Sensorsignale einzulesen.

- Verwendete I²C-Adressen: 0x6E und 0x6F

Motorsteuerung (Motoron):

Die Motoren werden über separate Motoron-I²C-Objekte angesteuert, wobei diese initialisiert, reinitialisiert und konfiguriert werden (u. a. Beschleunigungs- und Bremsparameter).

- Linker Motoron: Verwendet die Standardadresse.
- Rechter Motoron: Verwendet die Adresse 17.

IMU (Inertial Measurement Unit):

Initialisiert mittels der adafruit_icm20x Bibliothek über den I²C-Bus (über board.I2C())

Sensordaten und Filter:

Die Klasse definiert feste Sensorwinkel und einen maximalen Sensorradius. Für jeden Sensor wird ein LowPassFilter instanziiert, um die Rauschunterdrückung zu gewährleisten. Zusätzlich werden LowPassFilter für die jeweiligen Geschwindigkeitswerte der Räder.

Ein Unscented Kalman Filter (UKFEstimator) wird initialisiert. Dieser UKF ist funktionsfähig, wird jedoch in der aktuellen Anwendung nicht aktiv genutzt, da die Zustandschätzung des Roboters auch ohne den UKF ausreichend präzise ist.

Die Initialisierung des UKF umfasst:

- Erstellung von Sigma-Punkten (MerweScaledSigmaPoints)
- Festlegung von Prozess- und Messrauschen

- Definition von Bewegungs- und Messmodellen

Parameter Laden:

Die Parameter zur Umrechnung der ADC-Spannung in Abstände werden aus der angegebenen Datei geladen und in der Instanzvariablen `self.parameters` gespeichert.

Weiter Hardware Parameter:

Weitere Größen des Roboters, die hinterlegt werden, sind:

- `robot_radius`: Der Radius des Roboters
- `wheel_distance`: Der Abstand zwischen den Rädern
- `wheel_diameter`: Der Durchmesser der Räder
- `wheel_circumference`: Der Umfang der Räder
- `sensor_range`: Die Reichweite der Sensoren
- `gyro_w_bias`: Der Betrag des Drifts des Sensors der festgestellt wurde
- `ppr`: die Anzahl der Pulse pro Umdrehung der Encoder

Getter- und Setter-Methoden

`get_left_wheel_velocity(self)`

Gibt die Geschwindigkeit des linken Rads zurück.

Rückgabe:

- `velocity (float)` – Geschwindigkeit des linken Rads.

`get_right_wheel_velocity(self)`

Gibt die Geschwindigkeit des rechten Rads zurück.

Rückgabe:

- `velocity (float)` – Geschwindigkeit des rechten Rads.

`set_position_and_angle(self, x, y, theta)`

Setzt eine neue Position und Orientierung für den Roboter.

Parameter:

- `x, y, theta (float)` – Neue Position und Winkel.

`set_ukf_position_and_angle(self, x, y, theta)`

Setzt die UKF-geschätzte Position und Orientierung.

Parameter:

- `x, y, theta (float)` – Neue Position und Winkel.

Hauptmethoden

`get_position_and_angle(self)`

Gibt die geschätzte Position und den Winkel des Roboters zurück. Zur Schätzung der Werte muss `state_estimate` aufgerufen werden.

Rückgabe:

- `tuple(float, float, float, float)`: Position (`x, y`), Winkel, und tangentielle Geschwindigkeit.

`get_ukf_position_and_angle(self)`

Gibt die UKF-geschätzte Position und Orientierung zurück. Zur Schätzung der Werte muss `state_estimate_kalman` aufgerufen werden.

Rückgabe:

- `tuple(float, float, float)`: UKF-geschätzte Position, Winkel und tangentielle Geschwindigkeit.

`get_sensor_readings(self)`

Liest für alle Sensoren Spannungswerte über den ADC aus, rechnet diese mithilfe der Kalibrierungsparameter in Distanzen um und gibt sie als Liste zurück.

Rückgabe:

- `sensor_readings (list[float])`: Liste der Distanzen in cm.

`get_formatted_sensor_readings(self, sensor_readings, decimal_places=2)`

Formatiert die Sensordaten als Strings mit fester Breite und der angegebenen Anzahl von Dezimalstellen. Wird für die Konsolenausgabe verwendet.

Parameter:

- `sensor_readings (list[float])`: Liste der Sensordistanzen,
- `decimal_places (int, optional)`: Anzahl der Dezimalstellen (Standard: 2).

Rückgabe:

- Liste formatierter Strings.

`filter_sensor_readings(self, sensor_readings, time_step)`

Wendet einen Tiefpassfilter auf die Sensordaten an, um das Rauschen zu reduzieren.

Parameter:

- `sensor_readings` (list[float]): Rohdaten der Sensoren,
- `time_step` (float): Zeitdifferenz seit der letzten Messung.

Rückgabe: Gefilterte Sensordaten als Liste von float.

`get_imu_readings(self)`

Liest die Gyroskop-Daten über den I2C-Bus aus.

Rückgabe:

- Array (`gyro_x`, `gyro_y`, `gyro_z`): mit den gemessenen Gyroskopwerten.

`set_right_motor(self, speed)`

Diese Methode steuert den rechten Motor des Roboters. Sie übergibt den Geschwindigkeitswert an das `MotoronI2C`-Objekt, das den Motortreiber über den I²C-Bus ansteuert.

Parameter:

- `speed` (int): Der gewünschte Geschwindigkeitswert für den rechten Motor.

`set_left_motor(self, speed)`

Diese Methode steuert den linken Motor des Roboters. Sie übergibt den Geschwindigkeitswert an das `MotoronI2C`-Objekt, das den Motortreiber über den I²C-Bus ansteuert.

Parameter:

- `speed` (int): Der gewünschte Geschwindigkeitswert für den linken Motor.

`state_estimate(self, left_wheel_velocity, right_wheel_velocity, gyro_w, current_time, time_step)`

Berechnet die neue Position, Orientierung und Geschwindigkeit des Roboters anhand der aktuellen Encoderwerte und des Gyroskops. Diese Werte werden dann intern aktualisiert.

Parameter:

- `left_wheel_velocity`, `right_wheel_velocity` (float): Die Radgeschwindigkeiten,
- `gyro_w` (float): Gyroskop-Drehgeschwindigkeit,
- `current_time` (float): Aktueller Zeitstempel,
- `time_step` (float): Zeitdifferenz seit dem letzten Aufruf.

`state_estimate_kalman(self, left_wheel_velocity, right_wheel_velocity, gyro_w, current_time, time_step)`

Führt eine Zustandsschätzung mithilfe eines Unscented Kalman Filters (UKF) durch. Die Messung basiert auf Radgeschwindigkeiten und dem Gyroskop. Diese Werte werden dann intern aktualisiert.

Parameter:

- `left_wheel_velocity`, `right_wheel_velocity` (float): Die Radgeschwindigkeiten,
- `gyro_w` (float): Gyroskop-Drehgeschwindigkeit,
- `current_time` (float): Aktueller Zeitstempel,
- `time_step` (float): Zeitdifferenz seit dem letzten Aufruf.

Weitere Methoden

`load_parameters(self, filename)`

Lädt Kalibrierungsparameter aus einer Textdatei.

Parameter:

- `filename` (str): Pfad zur Parameterdatei.

Rückgabe:

- `parameters` (list[float]): Liste von Parametern (a, b, c, d, e) für die Spannungs-Distanz-Umrechnung.

`update_robot(self, x, y, theta, left_wheel_velocity, right_wheel_velocity, gyro_w, time_step)`

In dieser Methode ist das Modell implementiert welches von `state_estimate` aufgerufen wird, mit der die Position, Orientierung und Geschwindigkeit des Roboters basierend auf den aktuellen Radgeschwindigkeiten und dem gemessenen Winkelgeschwindigkeitswert berechnet wird.

Das zugrunde liegende Modell ist ein einfaches kinematisches Modell eines Differentialantriebsroboters.

Das Modell geht davon aus, dass der Roboter mit zwei unabhängig ansteuerbaren Rädern ausgestattet ist. Die zentrale Annahme ist, dass die tangentielle Geschwindigkeit des Roboters als Durchschnitt der linken und rechten Radgeschwindigkeit berechnet werden kann:

$$v = \frac{v_{\text{links}} + v_{\text{rechts}}}{2}$$

Die Änderung der Position in x- und y-Richtung erfolgt unter Verwendung der aktuellen Orientierung θ des Roboters und des Zeitschritts Δt :

$$x_{\text{neu}} = x + v \cdot \cos(\theta) \cdot \Delta t$$

$$y_{\text{neu}} = y + v \cdot \sin(\theta) \cdot \Delta t$$

Die Änderung der Orientierung wird direkt durch die gemessene Winkelgeschwindigkeit ω und den Zeitschritt bestimmt:

$$\theta_{\text{neu}} = \theta + \omega * \Delta t$$

Parameter:

- x (float): Aktuelle x-Position des Roboters.
- y (float): Aktuelle y-Position des Roboters.
- theta (float): Aktueller Orientierungswinkel des Roboters (in Radiant).
- left_wheel_velocity (float): Gemessene Geschwindigkeit des linken Rads.
- right_wheel_velocity (float): Gemessene Geschwindigkeit des rechten Rads.
- gyro_w (float): Gemessene Winkelgeschwindigkeit des Roboters (aus dem Gyroskop).
- time_step (float): Verstrichene Zeit seit der letzten Aktualisierung.

Rückgabe:

- Tuple: (x_neu, y_neu, theta_neu, v)
 - x_neu (float): Aktualisierte x-Position des Roboters.
 - y_neu (float): Aktualisierte y-Position des Roboters.
 - theta_neu (float): Aktualisierter Orientierungswinkel (in Radiant).

- `v (float)`: Berechnete tangentielle Geschwindigkeit des Roboters.

`update_robot_with_noise(self, x, y, theta, left_wheel_velocity, right_wheel_velocity, time_step)`

Aktualisiert die Roboterposition wie `update_robot`, fügt jedoch zusätzlich Systemrauschen hinzu. Dies wurde überwiegend in der Simulationsumgebung für testzwecken verwendet.

Parameter:

- `x (float)`: Aktuelle x-Position des Roboters.
- `y (float)`: Aktuelle y-Position des Roboters.
- `theta (float)`: Aktueller Orientierungswinkel des Roboters (in Radiant).
- `left_wheel_velocity (float)`: Gemessene Geschwindigkeit des linken Rads.
- `right_wheel_velocity (float)`: Gemessene Geschwindigkeit des rechten Rads.
- `gyro_w (float)`: Gemessene Winkelgeschwindigkeit des Roboters (aus dem Gyroskop).
- `time_step (float)`: Verstrichene Zeit seit der letzten Aktualisierung (in Sekunden).

Rückgabe:

- Tuple: (`x_neu`, `y_neu`, `theta_neu`, `v`)
 - `x_neu (float)`: Aktualisierte x-Position des Roboters.
 - `y_neu (float)`: Aktualisierte y-Position des Roboters.
 - `theta_neu (float)`: Aktualisierter Orientierungswinkel (in Radiant).
 - `v (float)`: Berechnete tangentielle Geschwindigkeit des Roboters.

`get_sensor_readings_with_noise(self, sensor_distances)`

Diese Methode liefert die Sensordaten des Roboters, wie sie durch die analoge Spannungsmessung ermittelt werden, zusätzlich versehen mit simuliertem Rauschen. Dies wurde überwiegend in der Simulationsumgebung für testzwecken verwendet.

Rückgabe:

- `sensor_readings (list[float])`: Liste der Distanzen in cm.

`handle_encoder_mode(self)`

Die Methode `handle_encoder_mode` bestimmt, wie die Radgeschwindigkeiten anhand der Encodersignale ermittelt werden. Dabei wird zwischen zwei Modi unterschieden:

- **Encoder Mode 0:**

Falls innerhalb eines definierten Zeitlimits (definiert durch `velocity_timeout`) kein Encoder-Impuls empfangen wurde, werden die Radgeschwindigkeiten (links und rechts) auf Null gesetzt.

- **Encoder Mode 1:**

In diesem Modus wird die Geschwindigkeit als Anzahl der Encoder-Impulse pro Zeiteinheit (Revolutions Per Second, RPS) berechnet.

Es wird die Anzahl der Impulse (`counter_left` bzw. `counter_right`) durch die Pulses Per Revolution (`ppr`) geteilt und durch den vergangenen Zeitschritt (`time_step`) normalisiert, um die RPS zu erhalten. Anschließend werden die RPS mit dem Umfang des Rades multipliziert, um die Geschwindigkeit in Metern pro Sekunde zu berechnen. Um Rauschkomponenten zu reduzieren, werden die Rohwerte durch einen Tiefpassfilter (`lpf_speed` bzw. `lpf_speed_right`) gefiltert. Schließlich werden die Encoder-Zähler zurückgesetzt, um bei der nächsten Berechnung wieder mit Null zu beginnen.

`convert_voltage_to_distance(self, voltage)`

Rechnet einen gemessenen Spannungswert eines IR-Sensors (von einem ADC-Kanal) in eine Distanz um, basierend auf den geladenen Kalibrierungsparametern.

Es wurde erwartet, dass sich die Intensität des Reflektierten Lichts mit dem Quadrat der Entfernung abnimmt und somit die Spannung, die vom Sensor, proportional zum Kehrwert des Quadrats der Distanz ist.

Um diese Beziehung mathematisch zu modellieren und die Spannung auf die Distanz abzubilden, wurde eine gebrochene rationale Funktion 3. Ordnung gewählt.

$$\text{distance} = \frac{a \cdot u^2 + b \cdot u + c}{d \cdot u + e}$$

Diese Methode hat sich als äußerst effektiv erwiesen und lieferte ausreichend gute Ergebnisse

Parameter:

- `voltage (float)`: Der vom ADC gelesene Spannungswert in cm.

Rückgabe:

- distance (float): Die berechnete Distanz.

`_update_count_left(self, gpio, level, tick)`

Inkrementiert den Zähler für den linken Encoder.

Parameter:

- gpio, level, tick: Hardware-spezifische Parameter (vom pigpio Callback übergeben).

`_update_count_right(self, gpio, level, tick)`

Inkrementiert den Zähler für den rechten Encoder.

Parameter:

- gpio, level, tick: Hardware-spezifische Parameter (vom pigpio Callback übergeben).

`_update_velocity_left(self, gpio, level, tick)`

Berechnet die Geschwindigkeit des linken Rads basierend auf der Zeit zwischen den Impulsen und aktualisiert diese mithilfe eines Tiefpassfilters.

Parameter:

- gpio, level, tick: Hardware-spezifische Parameter (vom pigpio Callback übergeben).

`_update_velocity_right(self, gpio, level, tick)`

Berechnet die Geschwindigkeit des rechten Rads basierend auf der Zeit zwischen den Impulsen und aktualisiert diese mithilfe eines Tiefpassfilters.

Parameter:

- gpio, level, tick: Hardware-spezifische Parameter (vom pigpio Callback übergeben).

`ukf_motion_model(self, state, dt)`

Implementiert das Bewegungsmodell (Prediction) für den UKF.

Parameter:

- state (np.array): Aktuelle Systemzustände (x, y, theta, omega, vl, vr, v).
- dt (float): Zeitschritt.

Rückgabe:

- Neue Zustände als `np.array (x , y , theta, omega, vl, vr, v)`.

`ukf_measurement_model(self, state)`

Implementiert das Messmodell für den UKF, das aus dem Zustand die erwarteten Messwerte berechnet.

Parameter:

- `state (np.array)`: Aktuelle Systemzustände (`x , y , theta, omega, vl, vr, v`).

Rückgabe:

- Erwartete Messwerte als `np.array (vl, vr, omega)`.

`get_power_percentage(self)`

Berechnet den Ladezustand des Akkus basierend auf der gemessenen Spannung.

Die Methode wird derzeit nicht aktiv in der Anwendung verwendet, da das Anschließen und Auslesen des Akkumoduls über den I²C-Bus zu Instabilitäten im Bus geführt hat. Um diese Funktionalität wieder zu verwenden, muss das INA219-Modul erneut an den Bus angeschlossen werden und im Konstruktor der Klasse muss die Initialisierung durch den Aufruf `self.aku = INA219()` erfolgen.

Rückgabe:

- Prozentsatz der Akkuladung (`float`).

Maus.py

Dieses Skript bildet den zentralen Einstiegspunkt zur Steuerung des Roboters und integriert alle wesentlichen Komponenten zur Hardwareansteuerung, Zustandsabschätzung sowie manueller und autonomer Regelung. Das Programm läuft auf einem Raspberry Pi 4 und ermöglicht es dem Benutzer, den Roboter entweder manuell oder im autonomen Modus zu betreiben. Die Auswahl des Modus sowie ein alternativer Parameterdateipfad können über Kommandozeilenargumente übergeben werden.

Das Skript **maus.py** initialisiert zunächst den Roboter, die Regelungsalgorithmen und das Ausgabe-Management. Anschließend wird in einer Endlosschleife (bis ein Abbruchsignal, z. B. STRG+C, empfangen wird) kontinuierlich der aktuelle Zustand des Roboters (Position, Orientierung, Geschwindigkeiten, Sensordaten) erfasst und ausgewertet. Auf Basis dieser Daten werden Sollwerte für die Radgeschwindigkeiten berechnet und an die Motoren weitergegeben. Im autonomen Modus übernimmt der „AutonomousController“ die Steuerung mithilfe des Pledge-Algorithmus, während im manuellen Modus die Eingaben über die Tastatur (über die Funktion `handle_user_input`) zur Steuerung verwendet werden.

Kommandozeilenparameter

Das Skript unterstützt zwei Kommandozeilenparameter:

- **-f, --filename** (str)
Gibt den Pfad zu einer alternativen Parameterdatei an, in der Kalibrierungswerte für die Umrechnung von Sensorspannungen in Distanzen hinterlegt sind. Wird dieser Parameter nicht übergeben, wird ein Standardpfad verwendet.
- **-a, --autonomous** (Flag)
Aktiviert den autonomen Modus. Ist dieser Parameter gesetzt, übernimmt der Roboter die autonome Steuerung mittels des Pledge-Algorithmus. Andernfalls erfolgt die manuelle Steuerung.

Steuerung

Für den Betrieb werden mehrere PID-Regler erstellt:

- `speed_pid_left` und `speed_pid_right` (PIDController):
Diese Regler steuern die Sollwerte der linken und rechten Radgeschwindigkeit.
 - Parameter:
 - `kp=700` (float)
 - `ki=3500` (float)
 - `kd=0` (float)

- Begrenzungen: $i_{\max}=550$, $d_{\max}=70$, $pid_{\min}=-100$, $pid_{\max}=600$

Es wurde experimentell getestet, dass die PID-Regler die Geschwindigkeiten des Roboters in einer Größenordnung regeln, die den Sollvorgaben entspricht.

Die exakte physikalische Korrektheit der Geschwindigkeiten ist jedoch nicht garantiert.

Daher sollten die Werte eher als ein Relatives Maß betrachtet werden.

Autonome Steuerung

Im autonomen Modus werden folgende Parameter konfiguriert:

- `init_base_speed` (float): Basisgeschwindigkeit des Roboters (0.12 m/s).
- `init_base_rotation_speed` (float): Basisdrehgeschwindigkeit (2.0 rad/s).
- `desired_distance` (float): Gewünschter Abstand zur Wand (3.5 cm).
- `sensor_activation_threshold` (float): Schwellenwert, ab dem die Sensoren als aktiv gelten (20cm).
- `diagonal_activation_threshold` (float): Schwellenwert für diagonale Sensoren (28 cm).
- `near_activation_threshold` (float): Schwellenwert zur Erkennung einer nahen Wand. (15 cm).

Diese Werte wurden so gewählt, dass sie ein möglichst robustes Durchfahren des Labyrinths gewährleisten. `init_base_speed`, `init_base_rotation_speed` und `desired_distance` haben einen unmittelbaren Effekt auf die Performance und das Verhalten des Roboters.

`sensor_activation_threshold`, `diagonal_activation_threshold` und `near_activation_threshold` beeinflussen nur die Wahrnehmung des Roboters und geringfügige Änderungen der Werte sollten keine ersichtlichen Verhaltensänderungen bewirken.

Für den ESC (Extremum Seeking Controller) werden folgende Parameter gesetzt:

- `dither_frequency` (float): Frequenz des Dithersignals
- `dither_amplitude` (float): Amplitude des Dithersignals
- `learning_rate` (float): Lernrate zur Anpassung des Steuerparameters
- `esc_angle_comparison_interval` (float): Zeitintervall zum Vergleich des ESC-Winkels
- `esc_angle_tolerance` (float): Toleranz für den ESC-Winkel

Hinweis: Der ESC-Regler wurde ursprünglich zur orthogonalen Ausrichtung in der Initialisierungsphase entwickelt, wird jedoch aktuell nicht verwendet.

Weitere Regler die Initialisiert werden.

- `point_distance_pid` (PIDController)

Regelt den Abstand nach vorne, beispielsweise wenn der Roboter sich einem bestimmten Punkt nähern soll.

- Parameter:
 - $kp = -1.2$
 - $ki = 0$
 - $kd = 0$
- Begrenzungen:
 - $pid_minmax = 0.5$

Es ist besonders wichtig, darauf zu achten, dass die Parameter für die Geschwindigkeitsregelung negativ gewählt werden. Dies hat den Hintergrund, dass bei einem zu großen Abstand der Roboter mehr Strecke vorwärts fahren soll und bei Überschreiten des Sollwertes eine Rückwärtsbewegung erfolgen soll. Wären diese Werte positiv, würde der Roboter sich genau gegenteilig verhalten.

- `wall_distance_pid` (PIDController)

Regelt den seitlichen Abstand zur Wand. Dieser Regler wird eingesetzt, um sicherzustellen, dass der Roboter einen konstanten Abstand zur Wand einhält, während er parallel an der Wand entlangfährt.

- Parameter:
 - $kp = 0.5$
 - $ki = 0.000$
 - $kd = 0.35$
- Begrenzungen:
 - $d_minmax = 0.029$, $pid_minmax = 12$, $pid_min = -0.2$

Der P-Anteil liefert stabile Regelungsergebnisse nur bei geringen Abweichungen. Bei größeren Abweichungen, wenn der Roboter zu stark auf die Wand zufährt, kann ein zu hoher P-Anteil zu einer Übersteuerung führen.

Ein moderater D-Anteil ist essenziell, da er die Neigung des Roboters ausgleicht, wenn er sich der Wand nähert. Wird der Roboter zu stark zur Wand geführt, ändert sich sein Winkel so, dass der Sensor aufgrund der veränderten Ausrichtung einen verlängerten Abstand misst, der nicht mehr den tatsächlichen Abstand widerspiegelt. Der D-Anteil stabilisiert den Regelkreis, indem er diese Winkelabweichungen entgegensteuert und somit eine parallele Ausrichtung zur Wand sicherstellt.

Die negative Begrenzung verhindert zusätzlich, dass der Regler zu stark übersteuert, wenn beispielsweise eine Lücke in der Wand vorhanden ist. In solchen Situationen könnte ein zu hoher negativer Reglerausgang dazu führen, dass der Roboter abrupt in die Wand hinein steuert.

- `diag_wall_distance_pid` (PIDController)

Funktion:

Regelt den Abstand zu einer diagonalen Wand. Dieser Regler hilft dabei, den Roboter in Situationen stabil zu halten, in denen der seitliche Abstand bei Außenecken nicht direkt gemessen wird.

- Parameter:
 - $k_p = 0.5$
 - $k_i = 0.000$
 - $k_d = 0.35$
- Begrenzungen:
 - $d_minmax = 0.029$, $pid_minmax = 10$, $pid_min = -0.2$

Zur Erhöhung der Flexibilität wurde der ursprünglich einheitliche Wandabstandsregler in zwei Regler aufgeteilt, einen für den geraden Seitenabstand (`wall_distance_pid`) und einen für den diagonalen Abstand (`diag_wall_distance_pid`)

- `angle_pid` (PIDController)

Dieser Regler wird zur Steuerung des Winkels verwendet, um die Ausrichtung des Roboters zu korrigieren.

- Parameter:
 - $k_p = 0.05$
 - $k_i = 0.00$
 - $k_d = 0.0$
- Begrenzungen:
- $pid_minmax = 1$, $pid_min = -0.5$

Die Parametrisierung dieses Reglers stellte eine besondere Herausforderung dar, da unterschiedliche Einsatzszenarien gegensätzliche Anforderungen an den Regler stellten:

- Optimierung für kleine Winkel:
Eine Konfiguration mit aktivem I-Glied erwies sich zunächst als hilfreich, da sie beim Anfahren kleiner Winkel unterstützte und ein schnelles Erreichen des

Sollwerts ermöglichte.

Bei größeren Winkelabweichungen führte das Integralglied jedoch zu einem übermäßigen Überspringen, da es vergangene Fehler zu stark akkumulierte.

- **Optimierung für große Winkel:**
Um das Überspringen zu vermeiden, wurde der Regler zunächst so eingestellt, dass er größere Winkelabstände regeln kann. In dem man die Parameter sehr schwach einstellt.
In diesem Szenario wurde der Regler allerdings zu schwach für kleine Winkel, was vermutlich auf Haftreibungsprobleme zurückzuführen, sodass der Roboter teils gar nicht zu drehen begonnen hat.

Das Problem wurde letztlich durch den Verzicht auf das I-Glied behoben. Stattdessen wurde eine lineare Rückführung der Drehgeschwindigkeit in die Stellgröße integriert. Diese Rückführung erfolgt im AutonomousController.

Ausgabe

Das Skript erstellt ein OutputManager-Objekt, das für die Konsolenausgabe und die Plot-Ausgabe (Echtzeit- und Batch-Plot) zuständig ist.

- Mit `start_console_output(auto)` wird ein `ConsoleOutput`-Objekt erstellt und (falls nicht manuell aufgerufen) in einem separaten Thread gestartet, um kontinuierlich den aktuellen Systemstatus anzuzeigen.
- Der Echtzeitplot wird zwar unterstützt, jedoch in der aktuellen Anwendung nicht genutzt, da er zu viel Latenz in das System bringt.

Ablauf der Hauptschleife

1. Initialisierung und Zeitmanagement:

- **Startzeit:**
Beim Eintritt in die Schleife wird der Startzeitpunkt (`start_Time`) ermittelt.
- **Endzeit:**
`duration`: Legt fest, wie lange das Programm laufen soll. In diesem Skript ist `duration = np.inf`, was einem unendlichen Lauf entspricht.
- **Zeitschrittberechnung:**
In jeder Iteration wird der aktuelle Zeitpunkt (`current_time`) relativ zum Start ermittelt. Der Zeitschritt (`time_step`) ergibt sich aus der Differenz zum vorherigen Zeitstempel (`last_time`).

2. Datenerfassung

- **Positions- und Zustandsdaten:**

Mittels `robot.get_position_and_angle()` werden die aktuelle Position (x, y), der Orientierungswinkel (theta) und die Geschwindigkeit (v) des Roboters erfasst.

- **Sensordaten:**

Über `robot.get_sensor_readings()` werden die aktuellen Sensordaten (z. B. Distanzen) abgerufen und anschließend durch `robot.filter_sensor_readings(sensor_readings, time_step)` von Rauschen befreit.

- **IMU-Daten:**

Die Gyroskopwerte werden über `robot.get_imu_readings()` ausgelesen, wobei der Wert für die horizontale Drehgeschwindigkeit (`gyro_w`) extrahiert wird.

3. Benutzereingaben und Modi

- **Manuelle Steuerung:**

Über die Funktion `io.handle_user_input(...)` werden Tastenanschläge verarbeitet, die:

- `base_speed` wird auf einen festen Wert gesetzt, der die Vorwärts- oder Rückwärtsbewegung des Roboters bestimmt.
- Der Parameter `angle_setpoint` dient nicht als Sollwinkel, sondern als Drehgeschwindigkeitskomponente. Er wird additiv bzw. subtraktiv zur Basisgeschwindigkeit kombiniert, um den Roboter nach links oder rechts zu drehen. Das bedeutet, dass eine positive Anpassung des `angle_setpoint` eine Drehung in die eine Richtung und eine negative Anpassung eine Drehung in die entgegengesetzte Richtung bewirkt.
- Den autonomen Modus (über Taste „a“) toggeln.
- Das Programm beenden (über Taste „c“).

- **Autonomer Modus:**

Ist der autonome Modus aktiviert (über Kommandozeilenparameter `-a` oder manuelle Eingabe), wird der Pledge-Algorithmus des `AutonomousController` aufgerufen, der auf Basis der Sensordaten und der aktuellen Roboterposition Sollwerte für die Radgeschwindigkeiten berechnet.

Ursprünglich wurde experimentell versucht, die Steuerung mittels inkrementeller Anpassung der vorwärts bzw. rückwärts tangentialen Geschwindigkeiten und eines Sollwinkels zu realisieren. Dies führte jedoch zu einer zu träge reagierenden Steuerung, welche im Manuellen Modus auch tendenziell intuitiv war. Daher wurde der Ansatz geändert und die Radgeschwindigkeiten direkt bestimmt.

4. Motoransteuerung:

Die PID-Regler sind das Herzstück der Motoransteuerung. Sie berechnen das Regelsignal anhand des Fehlers zwischen dem Soll- und Istwert der Geschwindigkeit. Insbesondere wird das Integralglied genutzt, um über einen längeren Zeitraum akkumulierte Fehler auszugleichen. Wird der Fehler jedoch sehr klein, kann ein zu stark integrierter Wert (Integralüberschuss) zu einem Überschwingen des Reglers führen.

Daher wird in solchen Fällen das Integralglied gezielt zurückgesetzt (mittels `set_integral()`), um Überschwingen entgegenzuwirken.

Da die Sensoren und Encoderdaten die Geschwindigkeiten als Beträge messen, da sich dies als weniger rauschempfindlich bewährte, ist es notwendig, mithilfe der `abs()`-Funktion stets mit positiven Werten zu arbeiten. Gleichzeitig muss die gewünschte Drehrichtung berücksichtigt werden.

Hierzu wird die Funktion `np.sign()` verwendet, die das Vorzeichen aus der Sollwertvorgabe für die berechneten Differenz liefert. Die Kombination aus `abs()` und `np.sign()` erlaubt es, den Regler so zu gestalten, dass er zwar auf den korrekten absoluten Fehler reagiert, aber dennoch das korrekte Vorzeichen, also die richtige Drehrichtung, für die Motoransteuerung liefert.

Die berechneten Werte für die linke und rechte Radgeschwindigkeit werden über die Methoden `robot.set_left_motor()` und `robot.set_right_motor()` an die Motoren übergeben.

Die Motoransteuerung erfolgt über die Motoron-Bibliothek, welche ausschließlich ganzzahlige Eingabewerte akzeptiert. Aus diesem Grund wird das von den PID-Reglern berechnete Regelsignal `final` mit der `int()`-Funktion in einen Integer umgewandelt, bevor es an die Motortreiber gesendet wird.

5. Zustandsschätzung:

Mittels `robot.state_estimate(...)` wird die Position, der Winkel und die tangentielle Geschwindigkeit des Roboters geschätzt und aktualisiert. Die tangentielle Geschwindigkeit wird derzeit nicht verwendet.

6. Output-Aktualisierungsfunktionen

Die Steuerungsschleife des Programms nutzt zwei wesentliche Funktionen zur Ausgabe von Daten: `out.update_console_output()` und `out.update_plot()`.

`out.update_console_output(...)`:

Diese Funktion aktualisiert die Konsolenausgabe. Sie sammelt alle relevanten Informationen,

wie aktuelle Roboterpositionen, Radgeschwindigkeiten, PID-Reglerwerte, Sollwerte (z. B. Basisgeschwindigkeit und Drehgeschwindigkeit) sowie Sensordaten. Im autonomen Modus werden zusätzlich spezifische Informationen wie der aktuelle Zustand des autonomen Controllers, der Folgesensor, der Pledge-Zähler und die Steuernachricht ausgegeben. Wenn sich die Steuernachricht ändert, wird die gesamte Konsolenausgabe (inklusive aller Informationen) in eine Log-Datei geschrieben.

`out.update_plot(...)`:

Diese Funktion übergibt die aktuellen Mess- und Sollwerte der Radgeschwindigkeiten an das RealTimePlotter-Objekt, das die Echtzeitvisualisierung übernimmt. Die Funktion sorgt dafür, dass die Diagramme laufend aktualisiert werden und so ein zeitlicher Verlauf der Radgeschwindigkeiten (sowohl Ist- als auch Sollwerte) dargestellt wird.

7. Abbruchbedingungen:

Die Schleife endet, wenn eine der folgenden Bedingungen eintritt:

- **Manueller Abbruch:**
Der Benutzer drückt in der Konsole **Strg+C** (KeyboardInterrupt).
- **Benutzereingabe über Tastatur:**
Wenn während der Schleifenlaufzeit die Taste "c" gedrückt wird, wird die Variable `close` auf `True` gesetzt und die Schleife beendet.
- **Zeitliche Begrenzung:**
Ist der voreingestellte Zeitwert (`duration`) erreicht, wird die Schleife beendet.
- **Ausnahmefehler:**
Tritt während der Ausführung der Schleife eine nicht behandelte Exception auf, wird die Schleife abgebrochen und eine Fehlerausgabe mit `traceback` erfolgt.

8. Ablauf nach Beendigung der Hauptschleife

Sobald die Hauptschleife (`main loop`) beendet wird – entweder durch Drücken von **STRG+C**, Erreichen des voreingestellten Zeitlimits (`duration`) oder aufgrund einer unerwarteten Exception – werden mehrere wichtige Maßnahmen ergriffen, um das System sicher herunterzufahren:

- **Motorstopp:**
Die Befehle zur Ansteuerung der Motoren werden aufgehoben, indem beide Motoren mit dem Wert 0 angesteuert werden. Dies stellt sicher, dass der Roboter sofort stoppt.

- **Konsolenausgabe beenden:**

Das OutputManager-Objekt ruft `stop_console_output()` auf, wodurch der zugehörige Thread (falls verwendet) beendet wird. So wird verhindert, dass weitere Konsolenausgaben oder Log-Einträge erfolgen.

- **Finale Visualisierung:**

Abschließend werden je nach gesetzten Flags entweder ein finaler Echtzeit-Plot oder ein Batch-Plot der aufgezeichneten Radgeschwindigkeiten angezeigt. Dies dient insbesondere dem Debugging und der Analyse des Fahrverhaltens.

- **Abschlussmeldung:**

Zum Schluss wird eine Meldung ("Messung beendet." bzw. "End of Program") ausgegeben, die den erfolgreichen Abschluss des Programms signalisiert.

Probleme und Verbesserungsmöglichkeiten

Im Entwicklungsprozess der autonomen Navigation des Roboters traten mehrere Herausforderungen auf, die in den Bereichen Sensorik, Signalverarbeitung, Antriebssteuerung und Zustandsabschätzung lagen. Im Folgenden werden die wesentlichen Probleme und potenzielle Verbesserungen zusammengefasst.

Ein Problem, das auftrat, war das Rauschen auf dem I2C-Bus, was sich besonders problematisch erwies. Durch das Entfernen des Akkumoduls vom Bus konnte die Instabilität vermieden werden.

Der I2C-Bus weist derzeit, wie die folgenden Abbildungen Zeigen, im Vergleich zu den anderen Signalen auch ein geringeres Rauschen auf.



Abbildung 6 I2C-Bus Data

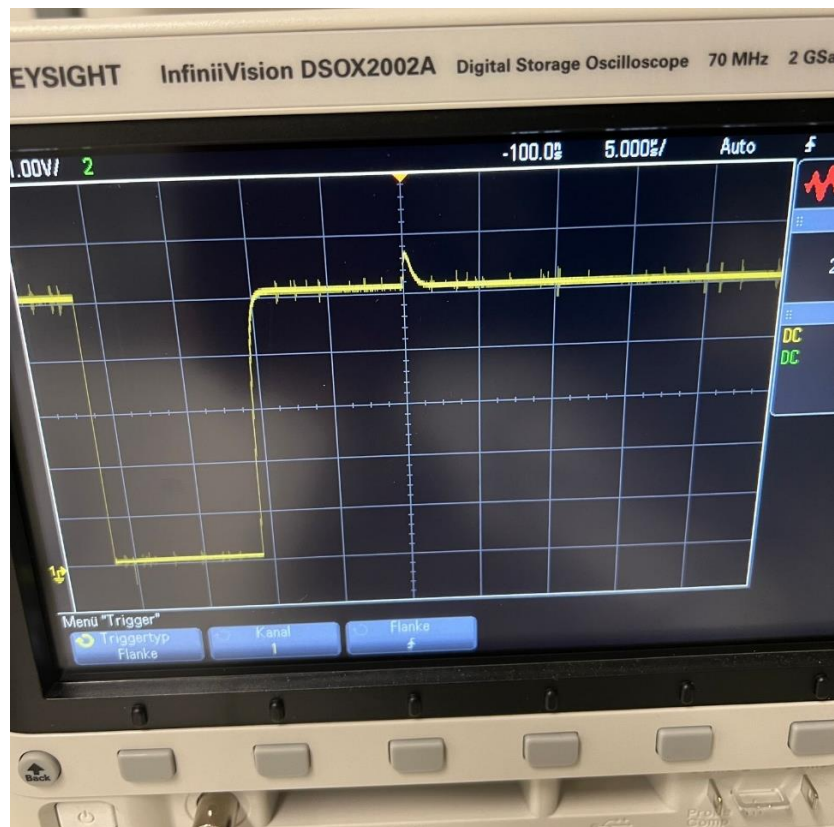


Abbildung 7 I2C-Bus Clock

Die Encoder-Signale waren stark verrauscht, was größere Probleme verursachte. Durch die Verfälschung der Geschwindigkeitsmessung wurde die Regelung der Motoren erschwert und die Zustandsschätzung unzuverlässig.

Das folgende Bild zeigt eine Messung eines Encoder-Ausgangs im Ruhezustand. Idealerweise sollte dieser konstant sein, jedoch zeigt er Abweichungen von nahezu einem Volt.

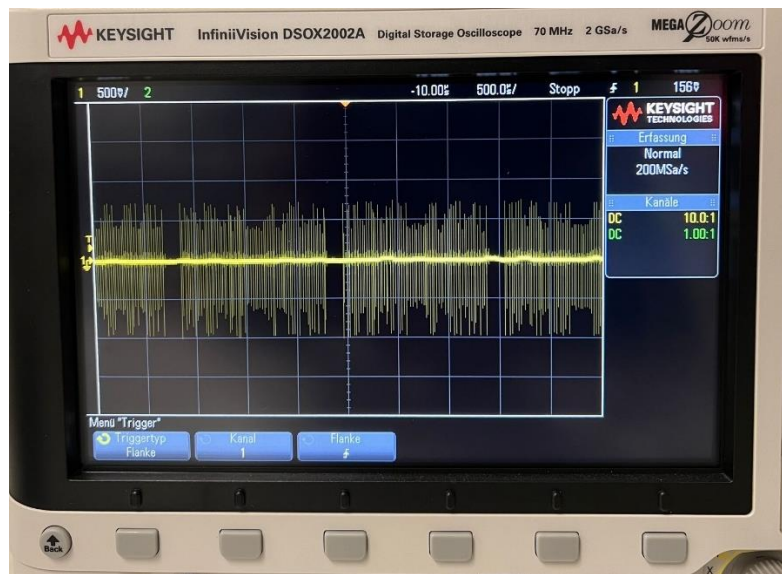


Abbildung 8 Encoder Signal

Das Problem wurde durch das ausschließliche Messen des Betrags und starke Filterung des Signals beherrschbar. Zusätzlich wurde die IMU hinzugefügt. Die Zustandsschätzung bestimmt nur Winkel und Position, wobei der Winkel die wichtigere Größe ist. Daher war es eine erhebliche Verbesserung, die Winkelgeschwindigkeit direkt mit der IMU messen zu können.

Auch die Sensorwerte sind stark vom Rauschen betroffen

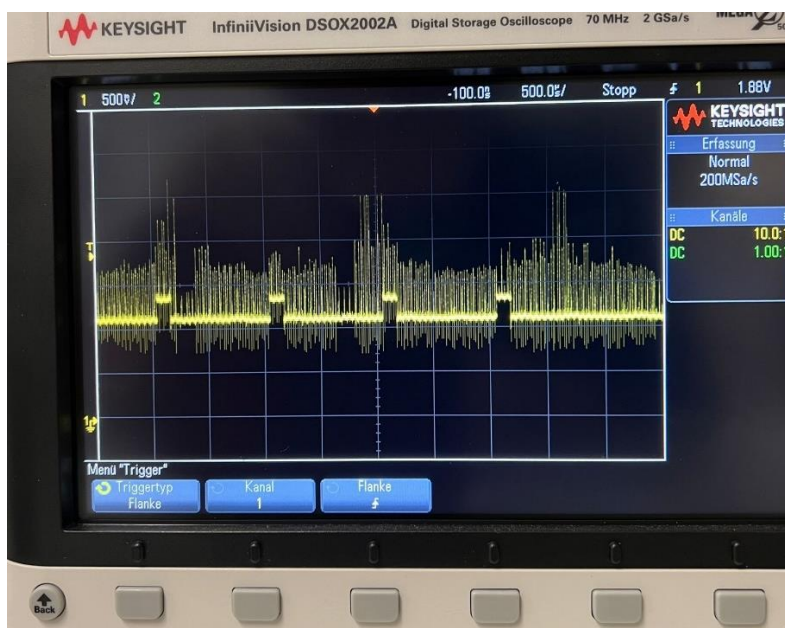


Abbildung 9 Sensor Signal

Durch die Filterung der Sensorsignale und die passende Wahl der Aktivierungsgrenzwerte konnten die nennenswerten Auswirkungen des Rauschens auf das Verhalten des Roboters jedoch weitgehend minimiert werden.

Weitere Verbesserungen des Rauschverhaltens wären dennoch sinnvoll. Hilfreich könnte es sein, wenn jedes Modul eine eigene Platine bekommt und die Leitungen, falls möglich, abgeschirmt werden.

Zusätzlich ist es so, dass man beim Hinzufügen oder Austauschen von Modulen einige Leitungen an- bzw. umstecken muss. Daher wäre es angenehm, wenn die Module mit integrierten Steckern ausgestattet würden, sodass sie beim Befestigen direkt an den I2C-Bus und die Stromversorgung angeschlossen werden.

Die maximale Leistung des Antriebs ist sogar größer als benötigt, daher besteht hier kein Verbesserungsbedarf. Problematisch ist eher eine zuverlässige sehr geringe Leistung, da bei einem geringen Steuersignal die Räder nur unzuverlässig zu drehen beginnen. Auch ist der Roboter sehr anfällig für Unebenheiten im Boden, sodass er gerne stecken bleibt. Daher ist eine allgemeine Überarbeitung des Antriebs sinnvoll, beispielsweise indem die Räder eine Federung bekommen.

Die Akkulaufzeit ist um ein Vielfaches größer als benötigt. Um Gewicht einzusparen, wäre es sinnvoll, einen kleineren Akku einzubauen.

Die Sensoren sind sehr scharf, was zu einer sehr präzisen Messung führt. Jedoch ergeben sich dadurch viele tote Winkel, die der Roboter nicht wahrnimmt. Auch ist der Roboter nicht in der Lage zu erkennen, wenn er gegen etwas stößt. Daher wäre das Hinzufügen weiterer Sensoren, wie zum Beispiel einem Bumperring, sinnvoll. Auch könnte man eine andere Anordnung der Sensoren in Erwägung ziehen, zum Beispiel zwei Sensoren parallel nebeneinander nach vorne zu haben, sodass man aus der Differenz der Abstandsmessungen den Winkel des Roboters zur Wand ermitteln kann.

Die Steuerlogik funktioniert derzeit nur für einen differenziellen Roboter. Es wäre schön, einen allgemeineren Ansatz zu finden, bei dem auch andere Robotertypen verwendet werden können. Dies wurde zwar zwischendurch versucht, indem Sollwinkel und tangential Geschwindigkeit statt der Radgeschwindigkeit vorgegeben wurden, dies führte jedoch leider nicht zu ausreichend guten Ergebnissen.

Auch ist es schade, dass der ESC-Regler nicht erfolgreich implementiert werden konnte, da vor allem das schnelle Steuern von kleinen Winkeln Probleme gemacht hat. Bei einer Verbesserung der Hardware könnte man dies vielleicht noch einmal angehen.

Der Kalman-Filter ist zwar funktionstüchtig, aber speziell die Systemrausch-Kovarianzmatrix war nicht bekannt. Hier könnte man eine Systemidentifikation durchführen, um an diese zu

gelangen. Dadurch würde man auch ein Modell erhalten, womit man eine analytische Parametrisierung der Regler durchführen und weitere modellbasierte Regelalgorithmen in Erwägung ziehen könnte. Es wurde zwar überlegt, mit dem einfachen Odometry-Modell einen Zustandsregler zu implementieren, es hat sich aber herausgestellt, dass sich dadurch lediglich ein P-Regler ergeben würde. Daher wurden die Regler auf einen PID-Regler erweitert.